

Algorithms and Data Structures (17/11/25)

Data Structure - Organising data inside memory for efficient processing along with operations like add, delete, search, etc. which can be performed on data.

e.g. stack - push/pop/top()

* Data can be stored in 2 ways:-

i) Contiguous memory location



ii) Non-Contiguous memory locations



* Physical Data Structures

- array
- linked list.

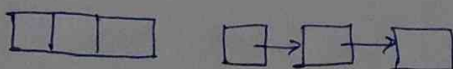
* Logical Data Structures - (Implemented using physical d.s.) (Tells relationship b/w the data)

- Stack
- Queue
- Tree.
- Heap
- Graph.

* Types of Data Structures

- Linear Data Structure (Basic)

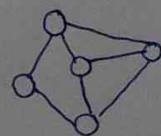
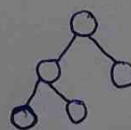
- data is organised linearly/sequentially
- data can be accessed sequentially
- e.g. Array, class/struct, Stack, Queue, Linked-List.



(Advanced)

- Non-Linear Data Structure

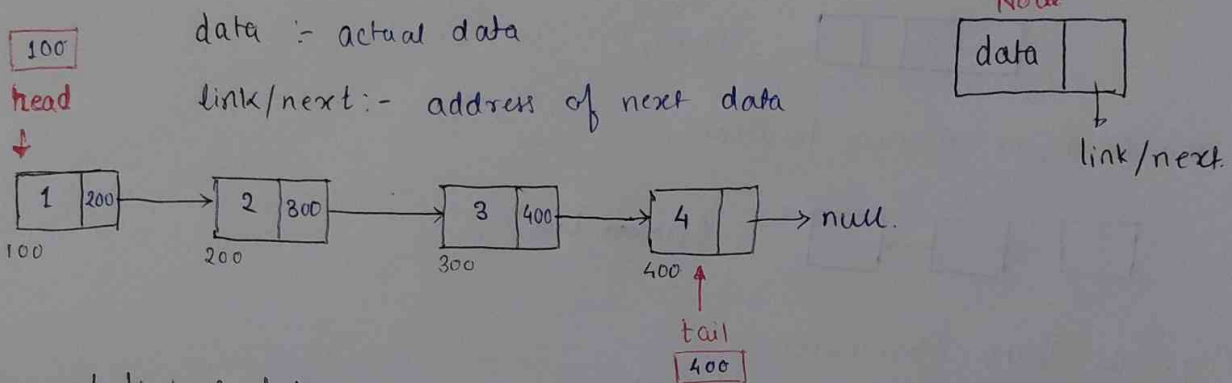
- data is organised in multiple levels (Hierarchy)
- data cannot be accessed sequentially
- e.g. Graph, Tree, Priority-Queue, etc.



* Hashtable / Hashmap can be considered as basic/advanced data structure as it can be implemented using any data structure (i.e. array or tree)

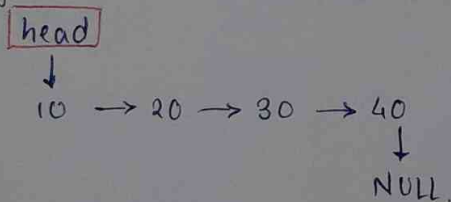
* Linked List (Linear D.S)

- Collection of similar type of data.
- every data keeps address of next data.
- element of linked list is called as Node.
- every node has two parts:-

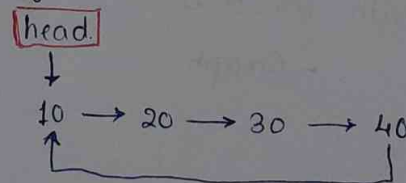


* Types of linked list

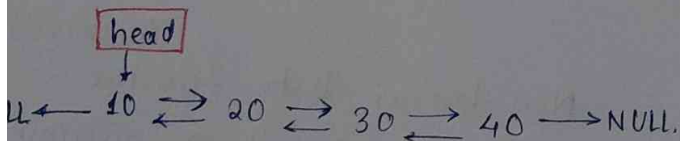
① Singly linear linked list.



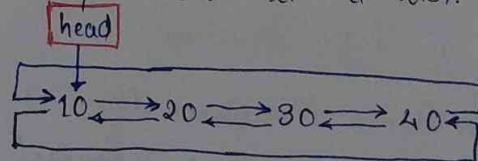
② Singly Circular linked list



③ Doubly linear linked list



④ Doubly Circular linked list.



* Implementation

Node:-

data: int, string, double, class, enum

next: reference of next node

```
class Node { ← Self Referential Class  
    int data;  
    Node next;  
}
```

class List {

Node head, tail;

int Size;

public List() { ... } → constructor

public void addNode() { ... }

public void deleteNode() { ... }

public void traverse() { ... }

public Node searchNode() { ... }

}

→ We need to access private fields of node class in methods of List class, without using getters & setters,

i.e. why we nest the Node class inside List class.

→ And, we make the Node class static to restrict the access of private fields of outer class into inner class.

So, updated class:-

class List {

static class Node {

}

Node head, tail,

}

* When can we make inner class static?

→ Only when the inner class does not depend on outer class.

i.e. it doesn't have any dependency of outer class to create object of inner class.

* Add a node at first

1) Create a newNode

```
(Node newNode = new Node(Value))
```

2) add first node's reference into newNode

```
(newNode.next = head)
```

3) move head to newNode.

```
(new head = newNode)
```

So, this code will add a node in an empty list as well as non-empty list, so, no need to check if list is empty or not.

* Traverse the Linked List

1) Create trav (Node) and start at head.

2) Visit / print data of current node.

3) move to next node.

4) repeat above 2 steps for every node.

```
(Node trav = head)
while (trav != null) {
    System.out.println(trav.data);
    trav = trav.next;
}
```

* We apply (trav != null) condⁿ when we want to traverse the complete list.

But, if we want the address/reference of last Node we use the (trav.next != null) condⁿ, here, in trav is pointing to last node.

* Add a Node at Last. (need to check for empty list)

1) Create a newNode

2) check if list is empty,

if empty - add newNode in head itself

3) if not empty,

a) traverse till last node.

b) add newNode into next of last node.

```
(Node newNode = new Node(val))
if (head == null)
    head = newNode;
else {
    Node trav = head;
    while (trav.next != null)
        trav = trav.next;
    trav.next = newNode;
}
```


* Delete the first node.

(move head $\rightarrow$ head.next.)

(And in java, Garbage collector will delete/release the occupied memory)

* Even if we have only one node, the same step will work.

* But if list is empty, we just return the head.

1) if list is empty

$\rightarrow$ return ~~head~~

2) else

- move head $\rightarrow$ head.next.

```
[ if (head == null)
    return
  else
    head = head.next. ]
```

* Delete the last node.

1) if list is empty

- return

2) if list has single node.

- make head equal to null.

3) if list has multiple node.

a) traverse till 2nd last node.

b) add null into next of 2nd last node.

```
[ if (head == null)
    return;
  else if (head.next == null) {
    head = null;
    return;
  }
  else {
    Node trav = head;
    while (trav.next.next != null)
      trav = trav.next;
    trav.next = null;
  } ]
```

* **Algorithm** :- Set of instruction to programmer
Written in human-understandable languages.

* Add node at a position

(Golden rule is $\rightarrow$ make before break) (i.e. create new links first & then break)

$\rightarrow$ travel till position-1 :-

Node trav = head;

for (i = 1; i < pos-1; i++)

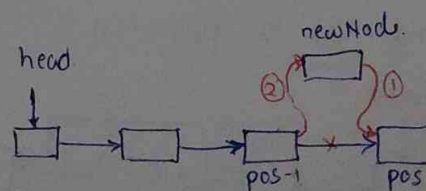
trav = trav.next.

$\rightarrow$ consider list is not empty :-

1) traverse till pos-1

2) add pos in the next of newNode;

3) add newNode into next of pos-1 node.



(Position = 4)

algo:-

- 1) create a new Node.
- 2) if list is empty, add newNode into head itself
- 3) if list is not empty
 - a) traverse till pos-1
 - b) add pos node into next of newNode.
 - c) add newNode into next of pos-1;

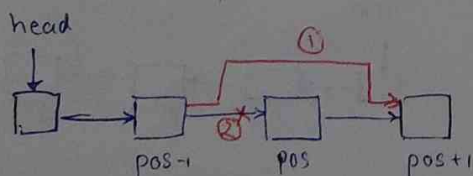
```
Node newNode = new Node(val);
if (head == null)
    head = newNode;
else {
    Node trav = head;
    for (int i = 1; i < pos-1; i++)
        trav = trav.next;
    newNode.next = trav.next;
    trav.next = newNode;
}
```

* Consider edge cases:-

- ① if position ≤ 1 ;
addFirst(value);
- ② if position $> (\text{size of linked list} + 1) \rightarrow$ throws error
add another condition to check if trav is on last node or not.
i.e. for (int i = 1; i < pos-1 && trav.next != null; i++)

```
Node newNode = new Node(val);
if (head == null)
    head = newNode;
else if (pos <= 1)
    addFirst(value);
else {
    Node trav = head;
    for (int i = 1; i < pos-1 && trav.next != null; i++)
        trav = trav.next;
    newNode.next = trav.next;
    trav.next = newNode;
}
```

* Delete node at a position.



- 1) if list is empty
- return
- 2) if list has only one node.
- head = null;
- return
- 3) if list has multiple nodes
- traverse till pos-1
- add pos+1 to next of pos-1
- ~~add next to~~

* handle edge cases

① if $pos == 1$
- `deleteFirst()`;

② if $pos < 1$
- `return`;

③ if $pos > \text{node count}$
- after traversing check ~~if~~,
if (`trav.next == null`)
 `return`;

($\because$ for all valid positions, the `trav` will ~~always~~ ^{never} be at last node)

```
if (head == null)
    return;
else if (pos == 1)
    deleteFirst();
else if (pos < 1)
    return;
else {
    Node trav = head;
    for (i = 0; i < pos-1 && trav.next != null; i++)
        trav = trav.next;

    if (trav.next == null)  $\rightarrow$  indicates an invalid position.
        return;
    trav.next = trav.next.next;
}
```

* Singly Circular Linked List

- * tail pointer is recommended in order to optimize add/delete operations.
- * we'll also maintain the size of linked list.

```
class List {  
    static class Node {  
        private int data;  
        private Node next;  
    }  
    private Node head;  
    private Node tail;  
    private int size;  
  
    public List () {  
        head = tail = null;  
        size = 0;  
    }  
  
    public boolean isEmpty () {  
        return head == null;  
    }  
  
    public void deleteAll {  
        head = null;  
        tail = null;  
        size = 0;  
    }  
}
```

- * to display all nodes data
 - 0) check if list is empty
 - 1) create trav and start at first node
 - 2) print current node data
 - 3) go on next node
 - 4) repeat above 2 steps for each node.

```
[ if (head == null) return;  
  Node trav = head;  
  do {  
      System.out.println(trav.data);  
      trav = trav.next;  
  }  
  while (trav != head);  
]
```


* Add a node at first

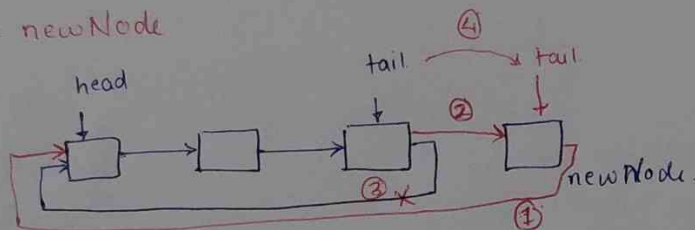
- 1) if list is empty
 - a) add newNode in head & tail
 - b) ~~add~~ make list circular
i.e. (tail.next = head)
- 2) if list is not empty
 - a) add head into next of newNode.
 - b) add newNode into next of tail
 - c) move head on newNode.

```
Node newNode = new Node(val)
if (head == null) {
    head = tail = newNode;
    tail.next = newNode; // or head or tail
}
else {
    newNode.next = head;
    tail.next = newNode;
    * head = newNode;
}
Size++;
```

* Add a node at last

Every thing remains, just we need to move tail instead of head.
i.e.

* tail = newNode



* Delete first node

- 1) if list is empty
- return;
- 2) if list has single node
- head = tail = null;
- 3) if list has multiple node
 - a) head = head.next;
 - b) tail.next = head;
- 4) Size--;

```
if (head == null)
    return;
else if (head == tail) {
    head = tail = null;
    return;
}
else {
    head = head.next;
    tail.next = head;
}
Size--;
```

→ here even when next is pointing to itself, but it's a self referencing reference & it's isolated from the outside world, so, GC will collect the object.

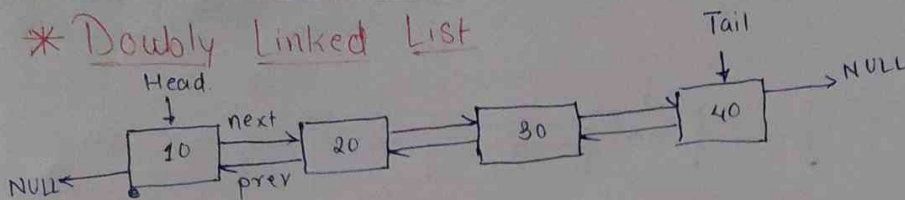
* Delete last node

- 1) if list is empty
- return
- 2) if list has single node
- delete that node
- 3) if list has multiple nodes →
 - a) traverse till 2nd last node
 - b) add first node into next of 2nd last node
 - c) move tail on 2nd last node.

→

```
[ if (head == null)
    return
  else if (head == tail)
    head = tail = null;
  else {
    Node trav = head;
    while (trav.next.next != head)
      trav = trav.next;
    trav.next = head;
    tail = trav;
  }
  Size--;
```

* Doubly Linked List



* Forward display

- Same as display singly linked list.

```

Node trav = head;
while (trav != null) {
    Sysout (trav.data);
    trav = trav.next;
}
  
```

* Backward display

- traverse from tail till trav != null.

```

Node trav = tail;
while (trav != null) {
    Sysout (trav.data);
    trav = trav.prev;
}
  
```

* Add node First

- 1) Create a newNode.
- 2) if list is empty
 - head = tail = newNode
- 3) if list is not empty
 - add head into next of newNode.
 - add newNode into prev of head.
 - move head to newNode.

```

Node newNode = new Node(val);
if (head == null)
    head = tail = newNode;
else {
    newNode.next = head;
    head.prev = newNode;
    head = newNode;
}
Size++;
  
```

* Add node at last

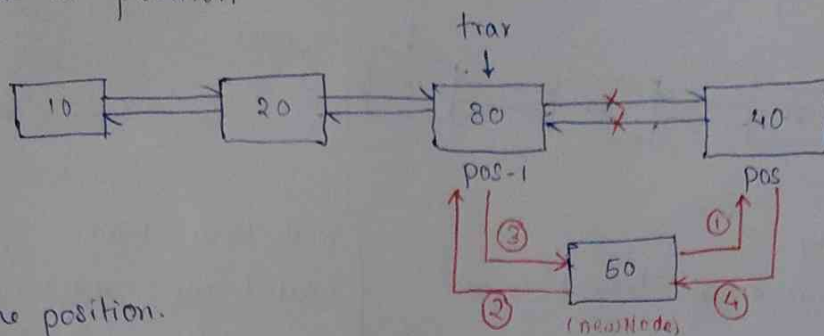
Similar ① & ② steps, and

- in 3) if list is not empty
- add newNode in next of tail.
 - add tail in prev of newNode.
 - move tail on newNode

```

Node newNode = new Node(val);
if (head == null)
    head = tail = newNode;
else {
    tail.next = newNode;
    newNode.prev = tail;
    tail = newNode;
}
Size++;
  
```

* Add a node at position



① ③ - create
② ④ - break

0) validate the position.

1) create a newNode

2) if list is empty

- head = tail = newNode

3) if list is not empty

3) if (pos == 1)

- addFirst(val)

4) if (pos == size + 1)

- addLast(val)

5) else {

- traverse till pos - 1

- add pos to next of newNode

- add pos - 1 to prev of newNode

- update pos prev to newNode

- update pos - 1 next to newNode.

```

if (pos < 1 || pos > size + 1)
    return;
if (head == null)
    head = tail = newNode;
if (pos == 1) {
    addFirst(val);
    return;
}
if (pos == size + 1) {
    addLast(val);
    return;
}
else {
    Node trav = head;
    for (i = 1; i < pos - 1; i++)
        trav = trav.next;
    newNode.next = trav.next;
    newNode.prev = trav;
    trav.next.prev = newNode;
    trav.next = newNode;
}
size++;
    
```

* Delete First

- ~~return~~ if list empty

- return

- if only one node

- move head & tail to null

- return

- if multiple node

- move head to next

- make prev of 2nd node to null

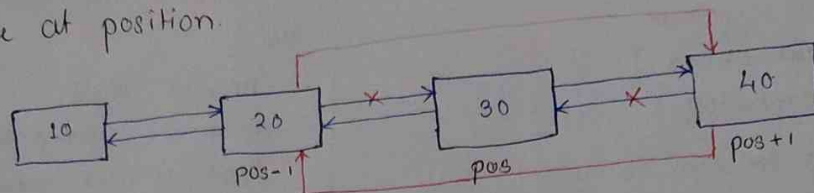
```

if (head == null)
    return;
if (head == tail)
    head = tail = null;
    return;
else {
    head = head.next;
    head.prev = null;
}
    
```


* Delete Last node

- Similar to deleteFirst, only we have to move tail pointer/ref to its previous node and then update the next of last 2nd node to be null.

* Delete node at position



- 1) validate the position
 - if ($pos < 1$ || $pos > size$)
return;
- 2) if ($pos == 1$)
return;
- 3) if ($pos == size$)
deleteLast();
- 4) if ($pos == size$)
deleteLast();
- 5) ~~else~~ if position is valid.
 - traverse till pos
 - $trav.prev.next = trav.next$
 - $trav.next.prev = trav.prev$

```

if (pos < 1 || pos > size || head == null)
    return;
if (pos == 1)
    deleteFirst();
    return;
if (pos == size)
    deleteLast();
    return;
else {
    Node trav = head;
    for (i = 1; i < pos; i++)
        trav = trav.next;
    trav.prev.next = trav.next;
    trav.next.prev = trav.prev;
}
    
```

* Doubly Circular Linked List

* Forward display

```

if (head == null)
    return;
Node trav = head;
do {
    Sysout(trav.data);
    trav = trav.next;
} while (trav != head);
    
```

// 10 20 30 40

* Backward Display

```

if (head == null)
    return;
Node trav = head;
do {
    Sysout(trav.data);
    trav = trav.prev;
} while (trav != head.prev);
    
```

// 40 30 20 10

* Add node at first

0) create a newNode

1) if list is empty

- head = tail = newNode
- make it circular
(tail.next = head;)
(head.prev = tail;)

2) if list is not empty

- add head in next of newNode.
- add ~~newNode~~ tail in prev of newNode.
- update next of tail with newNode
- update prev of head with newNode.
- move head to newNode

```
Node newNode = new Node(val);
```

```
if (head == null)
```

```
head = tail newNode;
```

```
nn.next = nn.prev = nn;
```

```
else {
```

```
nn.next = head;
```

```
nn.prev = head.prev; → tail
```

```
head.prev.next = nn newNode;
```

```
head.prev = nn;
```

```
* head = nn;
```

```
}
```

* Add node at last

Everything same, just at last instead of moving head to new node we move tail. ~~prev~~ to new node.

i.e. * head.prev = nn;

* Delete First

1) if list is empty

return;

2) if single node

head = null;

3) if list has multiple node.

- add 2nd node in next of last node.
- add last node in prev of 2nd node.
- move head to 2nd node

```
if (head == null)
```

```
return
```

```
if (head == head.next)
```

```
head = null;
```

```
else {
```

```
head.prev.next = head.next;
```

```
head.next.prev = head.prev;
```

```
head = head.next;
```

```
}
```

* Delete Last

1) & 2) steps are same

3) if list has multiple node

- add head into next of 2nd last node
- add 2nd last node in prev of head.

~~the example~~

```
else {
```

```
head.prev.prev.next = head;
```

```
head.prev = head.prev.prev;
```

```
}
```

Array

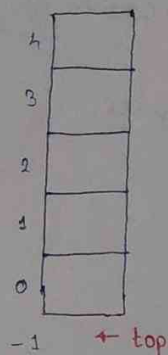
- Array space inside memory is contiguous.
- Array cannot grow/shrink at runtime.
- Random access of elements is allowed.
- Insert or delete, requires shifting of array elements.
- Array needs less space.

Linked List

- Linked List space inside memory is not contiguous.
- Linked List can grow/shrink at runtime.
- Random access of elements is not allowed.
- Insert or delete, doesn't require shifting of elements.
- Linked list needs more space.

* Stack

- linear data structure which has only one end - (top)
- data is inserted & removed from only one end - (top)
- Stack works on principle of - Last in First out / LIFO
- using array to create a fixed size stack



- top always points to last added/inserted data
- in insert data, we move top to a valid index and insert/assign the data in stack/array.
- to delete data from stack just decrement the top & when we again insert any data, the previous data will get overwritten with new data.

* Operations

① Push

- a) reposition top ($top++$)
- b) add data/value at top index

② Pop

- a) reposition top ($top--$)

③ Peek

- a) read data/value at top index.

* Conditions

① isEmpty

- if $top == -1$.

② isFull

- if $top == \text{Size} - 1$

* To implement stack using a linked list, we can choose to perform add/remove from any one end, then it'll behave like a stack.

① Add First / Delete First

② Add Last / Delete Last

* Stack Class

```
class Stack {
```

```
    private int arr[];
```

```
    private final int size;
```

```
    private int top;
```

```
    Stack (int size) {
```

```
        size = size;
```

```
        arr = new int [Size];
```


* Linear Queue

- linear data structure that has two ends - front & rear
- Data is inserted from back and removed from front end.
- Queue works on the principal of "First In First Out / FIFO".

* Operations:-

1) Push/enqueue

- a) reposition rear (rear++)
- b) add data/value at rear index

3) Peek

- a) read data from front-end.

2) Pop/dequeue.

- a) reposition front (front++)

* Conditions:-

1) isEmpty()

- a) if (rear == front)

2) isFull()

- a) if (rear == size-1)

* Implementation:-

```
Public class Queue {
```

```
    private int arr[];
```

```
    private int front, rear;
```

```
    private final int SIZE;
```

```
    public Queue (int size) {
```

```
        SIZE = size;
```

```
        arr = new int[SIZE];
```

```
        front = rear = -1;
```

```
    }
```

```
    void push (data) {
```

```
        if (isFull())
```

```
            return
```

```
        else
```

```
            rear++;
```

```
            arr[rear] = data;
```

```
    }
```

```
    int pop () {
```

```
        if (isEmpty())
```

```
            return 0;
```

```
        else
```

```
            temp = arr[front++];
```

```
            return temp;
```

```
    int peek () {
```

```
        if (isEmpty())
```

```
            return 0;
```

```
        else
```

```
            return arr[front];
```

* Here major problem is, the Queue is not getting utilised efficiently even after the space is available.

→ So, to resolve this, we reassign the rear & front to initial positions when queue is empty.

```
int pop () {  
    if (isEmpty())  
        return 0;  
    else {  
        temp = arr[front + 1];  
        front++;  
        if (rear == front)  
            rear = front = 0;  
        return temp;  
    }  
}
```

(But this is possible only when Queue is fully empty. Only then we reassign front & rear).

So, to tackle this we use Circular Queue.

* Circular Queue

We need to make changes in the size increment of front & rear.

i.e. $\text{front} = (\text{front} + 1) \% \text{size}$

$\text{rear} = (\text{rear} + 1) \% \text{size}$

* isEmpty()

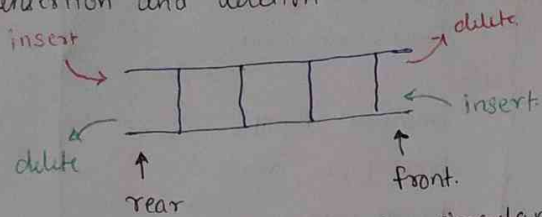
if (front == rear && rear == -1)

* isFull()

if ((front == -1 && rear == size - 1) ||
(front == rear && rear == -1))

Double Ended Queue

→ insertion and deletion is allowed from both the ends.



→ This queue is implemented by circular or doubly linked list.

→ Operations

- | | |
|-----------------|-------------|
| 1) pushfront(); | 5) getfront |
| 2) push rear(); | 6) getrear |
| 3) popfront(); | 7) isfull |
| 4) poprear(); | 8) isEmpty |

→ Deque Implementation.

```
class Deque {  
    static class Node {  
        int data;  
        Node head;  
        Node tail;  
        Node (int val) {  
            data = val;  
            head = tail = null;  
        }  
    }  
    private Node front;  
    private Node rear;  
    public Deque () {  
        front = rear = null;  
    }  
    public boolean isEmpty () {  
        return front == null && rear == null;  
    }  
    public int getfront () {  
        return front.data;  
    }  
    public int getrear () {  
        return rear.data;  
    }  
}
```

* Push front

- create a newnode
- if list is empty
 - front = rear = newnode
- if not empty
 - a) add front in next of newnode
 - b) add newnode in prev of front.
 - c) move front to newnode;

```
Node nn = new Node(val);  
if (front.isEmpty())  
    front = rear = nn;  
else  
    nn.next = front;  
    front.prev = nn;  
    front = nn;
```

* Push Rear

1st & two steps same & in 3rd we need to add element at last, i.e.

```
else  
    newnode.prev = rear;  
    rear.next = newnode;  
    rear = newnode;
```

(only code update)

* Pop front.

- check if empty .if yes
 - return -1
- get backup of data
- else
 - if list has single node
 - front = rear = null;
 - else
 - front = front.next;
 - front.prev = null

```
if (isEmpty())  
    return -1;  
int temp = front.data;  
if (front == rear)  
    front = rear = null;  
else  
    front = front.next;  
    front.prev = null;  
return temp
```

* Pop rear

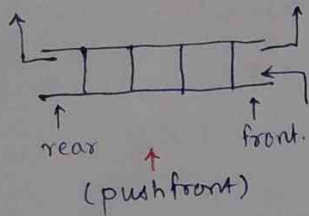
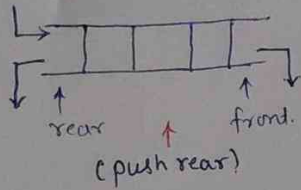
here also only last condition needs to be updated, i.e. (and get data from rear)

```
int temp = getRear();
```

```
else {  
    rear = rear.prev;  
    rear.next = null;  
}  
return temp;
```

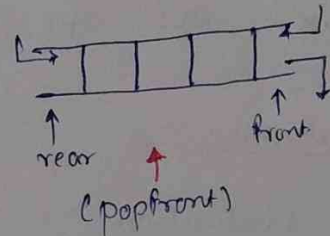
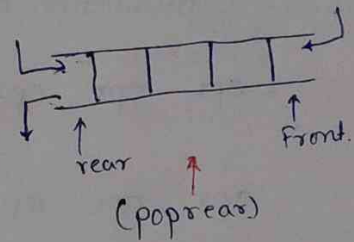

→ Input Restricted Queue

→ One input end is ^{closed.} ~~queue~~.



→ Output Restricted Queue

→ One output end is closed.



Applications of Queue

- in token system
- in network systems to
- in traversal of advanced data structures
- stack & queue can be efficiently used implemented using deque.
- can be used to check palindrome (as string, in linked list) using deque.
- can be also used to implement undo/redo operations using deque
- to implement ready queue in OS
- Browser history.

Uses of Stack

- lexical analysis in compilers
- Parenthesis Balancing
- store function Activation records
 - ↳ Control the flow of program
- traversal in advanced data structures
 - DFS → Stack
 - Preorder/Inorder/Postorder
- Expression conversion & Evaluation
 - (infix → postfix)
 - (prefix → infix)
 - (postfix → prefix)

* Expression conversion to evaluation

* Postfix Evaluation pseudocode.

postfixEval (expr) {

Stack <Integer> st = new Stack<>();

for (i = 0; i < expr.length; i++) {

char ele = expr.charAt(i);

if (Character.isDigit(ele))

st.push(ele - '0');

else {

int op1 = st.pop();

int op2 = st.pop();

int res = calculate(op1, ele, op2);

st.push(res);

}

}

return st.isEmpty() ? st.top : 0;

}

(in prefixEval)

→ for (i = length - 1; i >= 0; i--)

→ int op1 = st.pop();

→ int op2 = st.pop();

③ Infix to Postfix expression.

- Process each element from left to right
- if element is operand
 - append it to the postfix expression
- if element is operand
 - if priority of topmost (operator) element is greater or equal to current element, pop topmost element from stack & append it to postfix expression.
 - repeat above step if required
 - push element to stack
- Pop all remaining elements from stack one by one and append it to the postfix expression.

(we need to assign the priorities ourselves by writing a method)

Priority

^ or \$

*, /, %

+, -

Anything else

↑ higher

↓ lower

Pseudocode

~~Postfix to Infix~~

Infix to Postfix (expr) {

Stack <Character> st = new Stack<>();

StringBuilder postfix = new StringBuilder();

for (i = 0; i < expr.length; i++) {

if (Character ele = expr.charAt(i);

if (Character.isDigit(ele))

st.push(ele);

else if (ele == '(')

st.push(ele);

else if (ele == ')') {

while (!st.peek() == '(')

postfix.append(st.pop());

st.pop();

}

else {

priority

(while (!st.empty() && (st.peek() >= priority(ele)))

postfix.append(st.pop());

st.push(ele);

}

} → for loop ends.

while (!st.empty())

postfix.append(st.pop());

return postfix.toString();

}

Step to change
in infix →
postfix

④ Infix to Prefix Conversion

① Reverse the infix expression

② Find the Infix → Postfix expression, with minor change in operator comparison, i.e., here we only check if an operator is greater (not equal) or not;

③ reverse the obtained postfix expression

PTO

while (!st.empty() && prio(st.peek()) > prio(ele))

* Analysis of Algorithm

- finding exact time and space of algorithm is not possible because it depends on external factors like:-

① time is dependent on type of machine (CPU), no. of processes running at that time

② Space is dependent on type of machine (architecture), data type

* Time :- Time required to execute the algo on the CPU (us, ms, s)

* Space :- Space required to execute the algo inside the memory (bytes, kb, mb, bits)

→ Approximate time and space complexity is always done.

→ Mathematical approach used to find space & time requirements of the algorithm is called as Asymptotic analysis.

→ Asymptotic analysis also tells about the behavior of the algorithm for different input or for change in sequence of input.

→ behaviour of algorithm is observed in 3 diff cases:-

1) Best case

2) Avg. case

3) Worst case

Notations →	Big O (O)	Omega (Ω)	Theta (Θ)
	(upper bound)	(lower bound)	(Avg bound)

→ Time Complexity

- Time is directly proportional to number of iterations of the loop used in algo.
- to find time complexity / requirement of algo. count the number of iterations of the loop,

①

```
for (i=0; i<n; i++)  
{  
    ...  
}
```

$T(n) = O(n)$

$T \propto n$

②

```
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        ...  
    }  
}
```

$T(n) = O(m \times n)$

$T \propto n^2$

$T(n) \approx O(n^2)$

```

③ int sum (int n1, int n2) {
    return n1+n2;
}

```

→ irrespective of input time required will be same all the time

→ time required will be constant

```

④ void printBinary (int n) {
    while (n > 0) {
        rem = n%2;
        n = n/2;
    }
}

```

→ $n = n, n/2, n/4, \dots$

$= n/2^0, n/2^1, n/2^2, \dots, n/2^{itr}$

$$\frac{n}{2^{itr}} = 1$$

$$n = 2^{itr}$$

$$itr \log 2 = \log n$$

$$itr = \frac{\log n}{\log 2} \Rightarrow itr \propto \frac{1}{\log 2} \log n$$

$$itr \propto \log n$$

```

⑤ for (i=0 → n) { → O(n)
    ...
}
for (i=0 → n) { → O(n)
    ...
}
TC = O(2n)

```

* Modification is happening inside loop, then

if modification is (+ or -) → TC → O(n) ← in terms of

if modification is (* or /) → TC → O(log n) ← in terms of

* Space Complexity

Total Space = Input Space + Auxiliary Space

↓
Actual Space of input.

↓
extra space ^{acc} required by variables which are needed to process the input.

	Stack	Queue	S Linked List
push.	O(1)	O(1)	
pop	O(1)	O(1)	
peek	O(1)	O(1)	

Questions

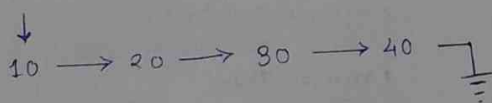
* Singly linked list - display reverse

- ① Start from head and push data into stack.
- ② move $trav = trav \rightarrow next$
- ③ repeat until $trav != null$
- ④ Start pointing top element & pop them until stack is empty.

OR

Recursive method

head



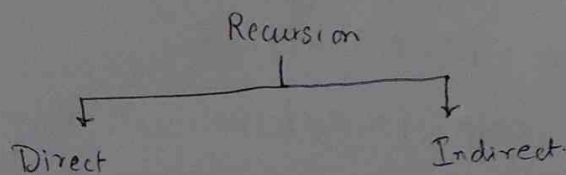
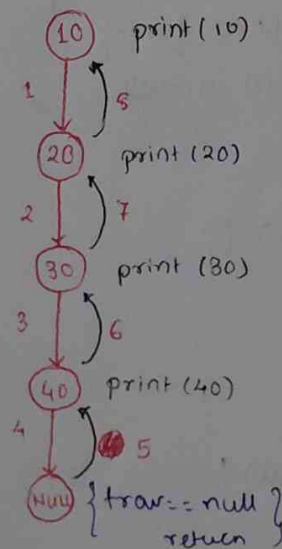
```
void rDisplay (Node trav)
{
    if (trav == null)
        return;
    rDisplay (trav.next);
    Sysout (trav.data);
}
```

Direct

* Two types of Recursion

- ① Tail Recursion
 - last statement of recursion is the recursion call.
- ② Head Recursion (Non-tail)
 - Recursion call is not the last statement.

Recursion Tree



```
fun() {
    ...
    fun();
}
```

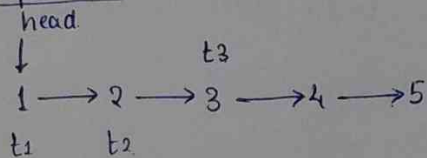
```
fun1() {
    ...
    fun2();
}
```

```
fun2() {
    ...
    fun1();
}
```

* Reverse the Singly linked list

Ex:- i/p: [1, 2, 3, 4, 5] 1 → 2 → 3 → 4 → 5
 o/p: [5, 4, 3, 2, 1] 5 → 4 → 3 → 2 → 1

① Take 3 pointers.



Repeat this steps until $t2 \neq \text{null}$ & before that check for if list is null.

$t3 = t2.\text{next}$ (store 3 for future) $t2.\text{next} = t1$ (reverse the link) $t1 = t2$ $t2 = t3$	} move them forward.
--	----------------------

Pseudocode :-

reverseList (Node root) {

Node t1 = null;

Node t2 = root;

Node t3;

while (~~no~~ t2 != null) {

 t3 = t2.next;

 t2.next = t1;

 t1 = t2;

 t2 = t3;

}

}

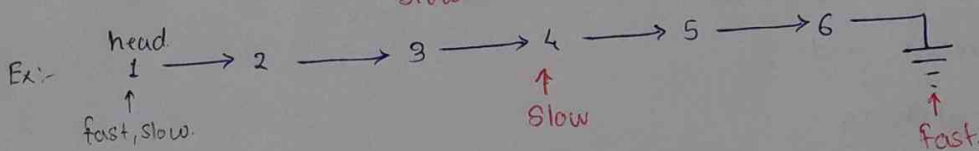
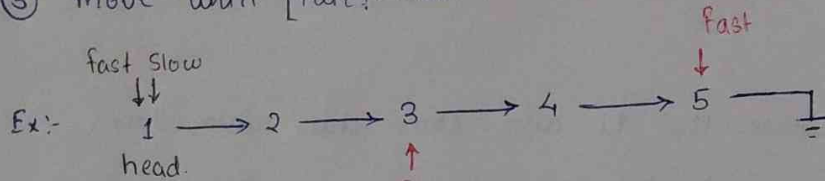
Middle of the Linked List (Fast & Slow Pointer method)
 (Floyd Cycle Detection Algo)

Ex:- [1, 2, 3, 4, 5] or [1, 2, 3, 4, 5, 6]

o/p:- [3, 4, 5]

o/p: [4, 5, 6]

- ② move fast by 2 nodes & slow by 1 node.
 ③ move until [fast != null && fast.next != null]



Node findMid () {

Node fast = head;

Node slow = head;

while (fast != Null && fast.next != null)

{

fast = fast.next.next;

slow = slow.next;

}

return slow;

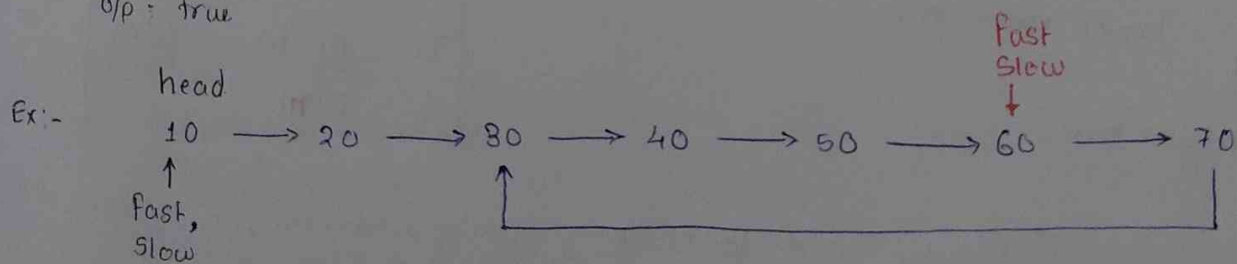
}

$$\left[\begin{array}{l} TC = O(n/2) \\ SC = O(1) \end{array} \right]$$

* Determine linked list cycle (Floyd cycle detection algo)

Ex:- ip: [3, 2, 0, -4] pos = 1

o/p: true



boolean hasCycle () {

Node slow = head, fast = head;

while (fast != null && fast.next != null) {

fast = fast.next.next;

slow = slow.next;

if (fast == slow)

{ return true;

} return false;

$$\left[\begin{array}{l} TC = O(n) \\ SC = O(1) \end{array} \right]$$

* Palindrome Linked List

Ex: [1, 2, 2, 1]



op: true.

- ① Take a stack and traverse the LL and store data into stack.
- ② now, start second traversal pop each element from stack and check with current node,
 - if not matching
return false.
 - continue

```
boolean isPalindrome () {
```

```
    Stack<int> st; Node trav = head;
```

```
    while (trav != null) {
```

```
        st.push(trav.data);
```

```
        trav = trav.next;
```

```
    }
```

```
    trav = head;
```

```
    while (!st.empty()) {
```

```
        if (st.top() != trav.data)
```

```
            return false;
```

```
        st.pop();
```

```
        trav = trav.next;
```

```
    }
```

```
    return true;
```

```
}
```

$$\left[\begin{array}{l} TC = O(2n) \approx O(n) \\ SC = O(n) \end{array} \right]$$

* Deque can also be used to checking for palindrome, i.e.

- ① push elements from one end (all elements)
- ② pop from both ends & check if both elem are same.
 - if not return false
 - else continue
- ③ at last if only one element remains, return true.

Searching

① Linear Search

- Traverse entire array
- Check for the key
 - if found return
 - else return -1;

② Binary Search

- ① Take key from user
- ② divide array into two parts (find middle element)
- ③ compare middle element with key
- ④ if key is matching
 - return index (mid)
- ⑤ if key is less than middle element.
 - Search key in left partition.
- ⑥ if key is greater than middle element.
 - Search in right partition
- ⑦ if key is not matching
 - return -1;

Pseudocode:-

```
int left = 0, right = n-1;
int mid =  $\frac{\text{left} + (\text{right} - \text{left})}{2}$ ;
while (left <= right) {
    if (num[mid] == key)
        return mid;
    if (num[mid] < key)
        left = mid + 1;
    else
        right = mid - 1;
}
return -1;
```

$$\begin{bmatrix} \text{TC} = O(\log n) \\ \text{SC} = O(1) \end{bmatrix}$$

* Recursive Binary Search

```
bs (left, right, key, arr)
```

```
{
    if (left > right)
        return -1;
    mid = (left + right) / 2;
    if (key == arr[mid])
        return mid;
```

```
    bs {
```

```
        if (key < arr[mid]) {
```

```
            return bs (left, mid-1, key, arr);
```

```
        }
```

```
    else {
```

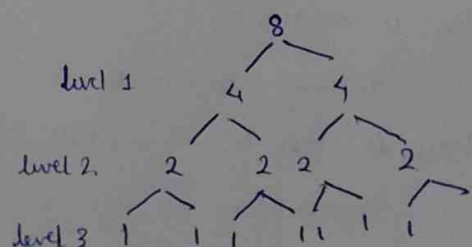
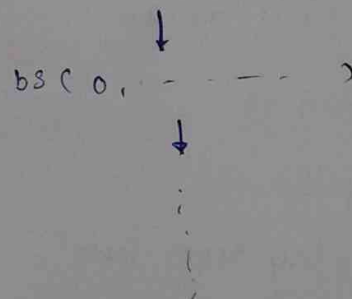
```
        return bs (mid+1, right, key, arr);
```

```
    }
```

```
}
```

Recursion Tree

```
bs (0, 25, 4, arr)
```



Time Complexity will depend on the level of in a recursion tree.

$$\therefore 2^d = n$$

$$d \log 2 = \log n$$

$$d = \frac{\log n}{\log 2}$$

$$\therefore \text{No. of iterations} = \frac{\log n}{\log 2} \rightarrow \text{constant}$$

$$\text{So, } [T.C = O(\log n)]$$

~~[S.C = O(1)]~~

$$[S.C = O(\log n)]$$

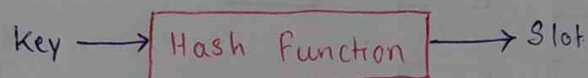
* Hashing

- Technique which allows to search data in constant time $O(1)$
- Can be used to search, insert, delete in constant average time $O(1)$.
- Implementation of hashing is known as hash table.
- Hash table is an array of fixed size, in which elements are stored in key-value pairs.

Array - Hash Table

Index - Slot.

- In hashtable, only unique keys are stored
- Every key is mapped with one slot of the table, and this is done with the help of hash function.



- ↳ ① should generate unique hash
② should be easy to compute.

- This follows associative access, i.e. data is kept in key-value pairs and data is accessed using key, and it is the fastest.

Size = 10

K, V		
8 - V1		
3 - V2		
10 - V3		
4 - V4		
6 - V5		
13 - V6		

10, V3	0
	1
	2
3, V2	3
	4
	5
6, V5	6
	7
8, V1	8
	9
	10

Hash Function

$$h(k) = k \% \text{Size}$$

$$h(8) = 8 \% 10 = 8$$

$$h(3) = 3 \% 10 = 3$$

$$h(10) = 10 \% 10 = 0$$

$$h(4) = 4 \% 10 = 4$$

$$h(6) = 6 \% 10 = 6$$

$$h(13) = 13 \% 10 = 3$$

Collision

Add - 1) slot = $h(k)$

2) $arr[slot] = \{key, value\}$

Search :- 1) slot = $h(k)$

2) return $arr[slot].value$

delete - 1) slot = $h(k)$

2) $arr[slot] = null$

* **Collision** :- Situation when 2 different keys, yield same slots

Collision handling techniques:-

① **Closed Addressing**

(Chaining) or

(Separate Chaining)

② **Open Addressing**

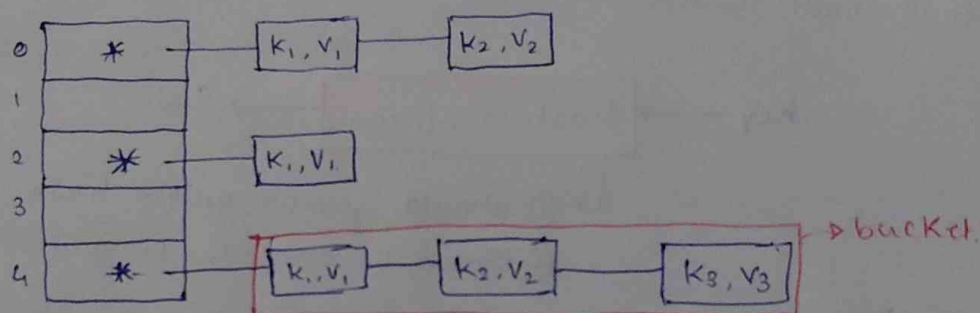
↳ a) linear probing

↳ b) quadratic probing

↳ c) double-hashing

* In closed addressing, we maintain a linked list at each slot and in that linked list {key, value} pairs are added.

i.e.



* Every linked list present on the slot, is called as **Bucket**.

* Disadvantage

- 1) key-value pairs are stored outside the table
- 2) Space requirement is more.
- 3) worst case time complexity maybe $O(n)$.

* Advantage

- 1) No restriction on number of key-value pairs

(implementation in pdf/code)

* Open addressing

→ Probing :- finding next free slot where key value pair will be kept
(Linear Probing) when collision will occur.

10, V3	0
	1
	2
3, V2	3
4, V4	4
5, V9	5
6, V6	6
	7
8, V1	8
	9
	10

$$h(K) = K \% \text{ size}$$

$$h(K, i) = [h(K) + f(i)] \% \text{ size}$$

$$(f(i) = i)$$

where, $i = 1, 2, 3, \dots$

(probe number)

$$* h(13) = 13 \% 10 = 3$$

$$1^{\text{st}} \text{ probe: } h(3, 1) = [3 + 1] \% 10 = 4$$

$$2^{\text{nd}} \text{ probe: } h(3, 2) = [3 + 2] \% 10 = 5$$

* Disadvantage:-

* Primary Clustering

- near key position table becomes bulky or crowded
- need to take long run of filled slots near key pos" to get empty slot

* Quadratic Probing:-

	10, V3	0
		1
		2
8, V1		3
3, V2		4
10, V3		5
4, V4		6
6, V5		7
13, V6		8
		9

Collision

$$h(K) = K \% \text{ size}$$

$$h(K, i) = [h(K) + f(i)] \% \text{ size}$$

$$f(i) = i^2$$

where, $i = 1, 2, 3, \dots$

can be anything quadratic

Advantage:-

① Primary Clustering is solved

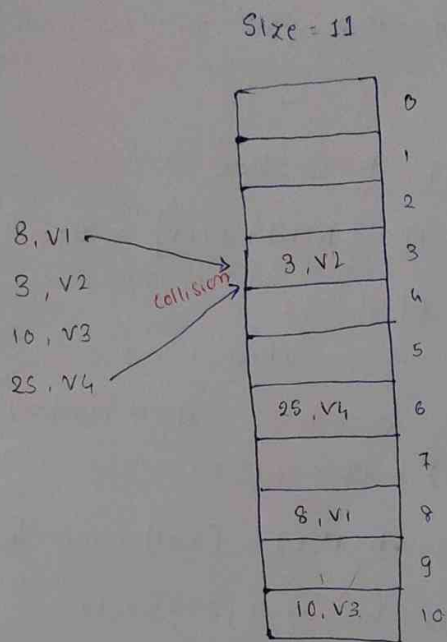
Disadvantage:-

① Secondary Clustering occurs:-

- need to take long run of filled slots "away" key position to get empty slot

② There is no guarantee that we'll get free slot for key-value pair.

* Double hashing



$$h_1(k) = k \% \text{size}$$

$$h_2(k) = 7 - (k \% 7)$$

$$h(k, i) = [h_1(k) + i * h_2(k)] \% \text{Size}$$

$$h_1(8) = 8 \% 11 = 8$$

$$h_1(3) = 3 \% 11 = 3$$

$$h_1(10) = 10 \% 11 = 10$$

$$h_1(25) = 25 \% 11 = 3$$

$$h_2 = 7 - (25 \% 7) = 3$$

$$\begin{aligned} 1^{\text{st}} \text{ probe} &= h(25, 1) = [3 + 1 * 3] \% 11 \\ &= 6 \% 11 = \underline{6} \end{aligned}$$

→ Here as the no. of slots gets filled & hash table gets filled more than 50%, then we increase the table size by double, this is called as.

Rehashing.

* Rehashing

$$\text{Load factor} = \frac{n}{N}$$

n = no. of elements (present in hash table)

N = no. of slots (total)

* Load factor ranges from $\{0 - 1\}$.

- if $n < N$:

Load factor < 1 - free slots are available

- if $n = N$:

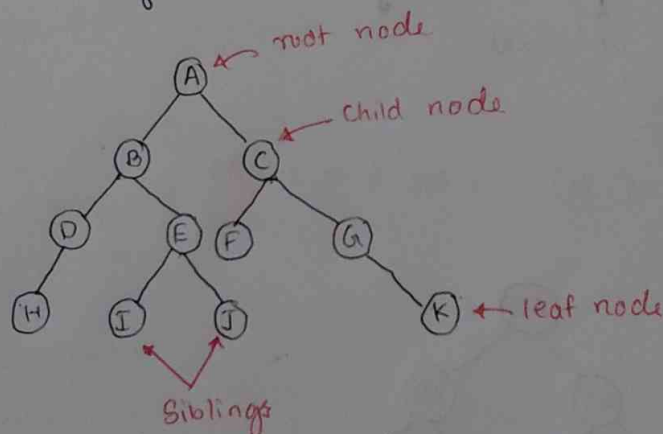
Load factor $= 1$ - free slots are not available

* In rehashing, whenever hash table will be filled more than 60% or 70% size of hash table is increased by twice.

* Existing key value pairs are remapped with updated size.

* Tree Data Structures

- Tree is a non-linear data structure, which is a finite set of nodes with one specially designated node is called as "root".
- Root is a starting point of the tree.
- All nodes are connected in Hierarchical manner (multiple levels)
- Parent Node:- having other child nodes connected
- child node:- immediate descendant of a node.
- Leaf Node:-
 - Terminal node of the tree.
 - Leaf node does not have child nodes.
- Ancestors:- all nodes in the path from root to that node.
- Descendants:- all nodes accessible from the given node.
- Siblings:- child nodes of the same parent.



- Degree of a node:- no. of child nodes for any given node.
- Degree of a tree:- maximum degree of any node in tree
- Level of a node:- indicates position of the node in tree hierarchy

Level of child = Level of Parent + 1.

Level of root = 0.

- Height of node:- number of links from node to longest leaf.
- Depth of node:- number of links from root to that node.
- Height of a tree:- maximum height of a node.
- Depth of a tree:- maximum depth of a node.

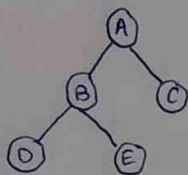
* A tree with zero nodes (is empty tree) is called as Null Tree.

Height of Null Tree is -1 .

* Tree can grow up to any level. and any node can have any number of childs.

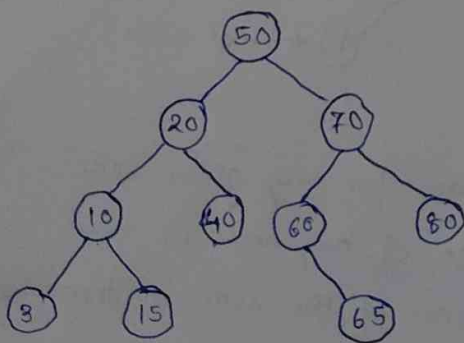
* Binary Tree

- Treeⁱⁿ which each node has maximum two childs nodes.
- Binary tree has degree 2. Hence its also called as 2-tree.



* Binary Search Tree

- Binary tree with left node containing smaller value and the right node contains the greater value always, to the parent node.
or equal.
- Searching is faster.
- Time complexity - $O(h)$ ~~aka~~ ($h \rightarrow$ height)



(max. ops in a bst are performed in $O(h)$ T.C.)

* Implementation of B.S.T

Node :-

data : data of a node

left : reference of left child

right : ——— right child

→ Class BST {

static class Node {

P int data;

P Node left;

P Node right;

} Node (data) {
 ~~data~~ = data;

→ constructor

public Node (val) {
 data = val;
 left = right = null;
}

Node root;

public BST (~~root~~) {

~~data~~ = val;
root = null;

}

public boolean isEmpty () {

return root == null;

}

public void deleteAll () {

root
~~root~~ = null;

}

}

~~void addNode (val) {~~

~~if (isEmpty()) {~~

~~root = new Node;~~

}

~~root = new Node;~~

* Add a node in BST

① Create a newNode with given data

② if BST is empty,

add newNode in root node itself

③ if BST is not empty

3.1) create a trav reference and start at root

3.2) if value is less than current node data

3.2.1) if left of current node is empty

// add newNode into left of current node

3.2.2) if left of current node is not empty

// go to left of current node

3.3) if value is greater than or equal to current node data.

3.3.1) if right of current node is empty

// add newNode into right of current node

3.3.2) if right of current node is not empty

// ~~add~~ go to one right of current node.

3.4) Repeat above two steps till newNode is not added.

→ Pseudocode

```
Node newNode = new Node(data);
```

```
if (isEmpty)
```

```
    root = newNode;
```

```
else {
```

```
    while (true) {
```

```
        Node trav = root;
```

```
        if (trav.data > value) {
```

```
            if (trav.right == null) {
```

```
                trav.right = newNode;
```

```
                break;
```

```
            } else trav = trav.right;
```

```
        } else {
```

```
            if (trav.left == null) {
```

```
                trav.left = newNode;
```

```
                break;
```

```
            } else trav = trav.left;
```

```
        }
```

```
    }
```

height of tree

TC = $O(h)$

Pre Order $\rightarrow$ V L R

In Order $\rightarrow$ L V R

Post Order $\rightarrow$ L R V

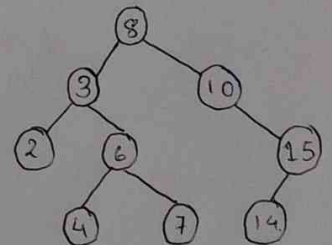
V - visit

L - left

R - right

* BST - Binary Search

- 1) Start from root
- 2) If key is equal to current node data return current node.
- 3) If key is less than current node data, search key into left sub tree of current node.
- 4) If key is greater or equal to current node data, search key into right sub tree of current node.
- 5) Repeat step 2-4 till leaf node.



Pseudocode:-

```
Node trav = head root;
```

```
While ( trav != null)
```

```
{ if ( trav.data == key)
```

```
    return trav;
```

```
    else if ( trav.data > key)
```

```
        trav = trav.left;
```

```
    else
```

```
        trav = trav.right;
```

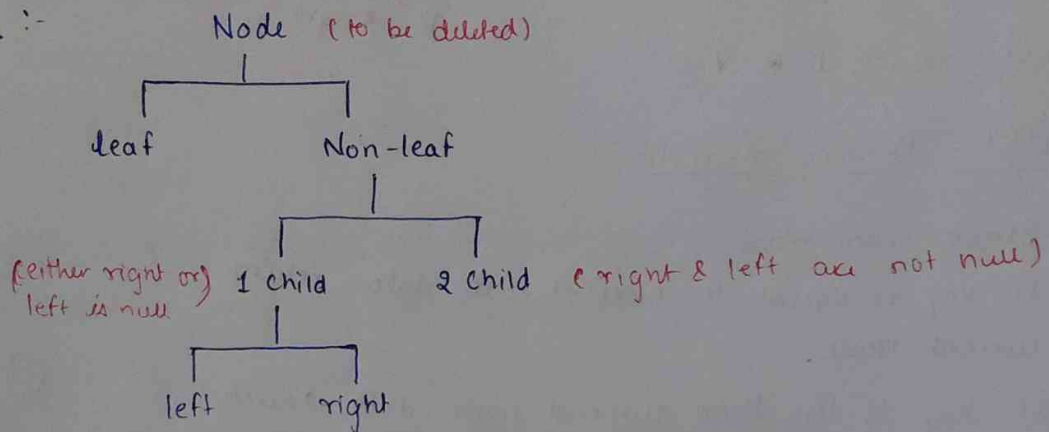
```
}
```

```
return null
```

$TC = O(\log_2 N)$ $\rightarrow$ height of tree

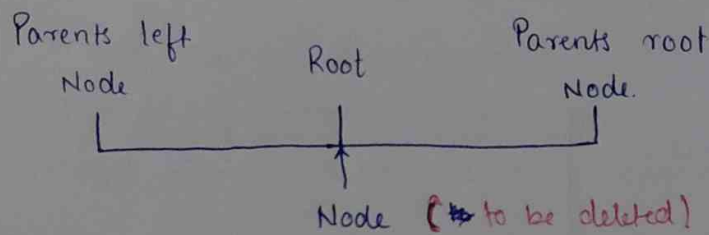
* Delete a Node

Possibilities :-



* There are 4 possibilities if we want to delete a node.

* And Other 3 possibilities of what that node is, i.e.



* We also need to keep the reference of Parent of the node to be deleted.

Algo

1) Search for a key along with its parent (take parent & trav references & just do, parent = trav;

Node trav = root, parent = null;

while (trav != null) {

if (key == trav.data)

break;

else if (trav.data > key) {

parent = trav;

trav = trav.left;

}

else {

parent = trav;

trav = trav.right;

}

2) if node is not found then return

```
if (trav == null)
    return;
```

3) if node is found delete it

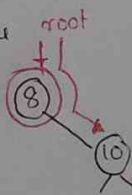
3.1) if node has 2 childs

3.2) if node has single child (right)

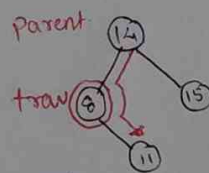
3.3) if node has single child (left)

→ delete node with single child (Right child)

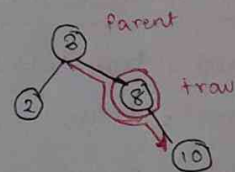
(a) root node



(b) Parents right



(c) Parents left



→ check if it has only right child.

→ if node is root

- update root by right child

→ if node is parent's left child

- update parent's left with right child.

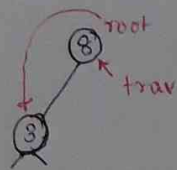
→ if node is parent's right child

- update parent's right ~~child~~ by right child.

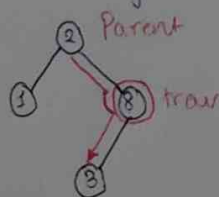
```
if (trav.left == null) {
    if (trav == root)
        root = trav.right;
    else if (parent.left == trav)
        parent.left = trav.right;
    else
        parent.right = trav.right;
}
```

→ delete node with single child (left child)

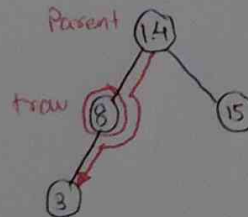
(a) root node



(b) Parents right



(c) Parent's left



→ check if it has only left child

→ if node is root

- update root with left child

→ if node is parent's left child.

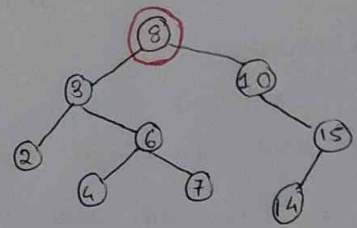
- update parent's left with left child.

→ if node is parent's right child

- update parent's right with left child

* delete node with two child node.

inorder $\rightarrow$ 2 3 4 6 7 ^{root} 8 10 14 15
 inorder predecessor $\uparrow$ $\uparrow$ inorder successor



→ These are the only 2 values with which we can replace $\textcircled{2}$ / root with.

→ And the inorder-~~predecessor~~^{predecessor} is the right-most leaf in left sub-tree,
and the inorder-successor is the left most leaf in right sub tree.

to find Predecessor

pred = trav.left; (trav = root)

parent = trav;

```
while ( pred.right != null ) {
```

parent: ~~answer~~, pred;

pred ~~new~~: pred. right;

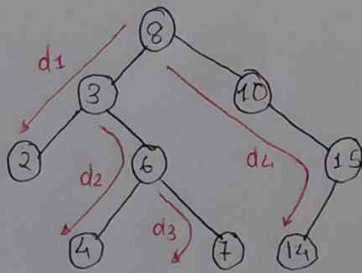
return true;

Algo.

- 8.3) a) find the predecessor along with its parent.
b) replace p root data with predecessor data
c) delete predecessor

* DFS Traversal (Depth first Search)

- 1) Push the root node on stack
 - 2) Pop one node from stack
 - 3) Visit (print) popped node
 - 4) If right exists, push it on stack
 - 5) If left exists, push it on stack
 - 6) While stack is not empty, repeat step 2 → 5.
- Sequence can be changed
a/c to the need.



Traversal :- 8 3 2 6 4 7 10 15 14

4
15
4
7
2
6
8
10
8

Pseudocode :-

dfsTraversal () {

Stack <Node> st = new Stack<>();

st.push(root);

while (!st.empty())

{ Node popped = st.pop();

print(popped.data)

if (popped.left != null)

st.push(popped.left);

if (popped.right != null)

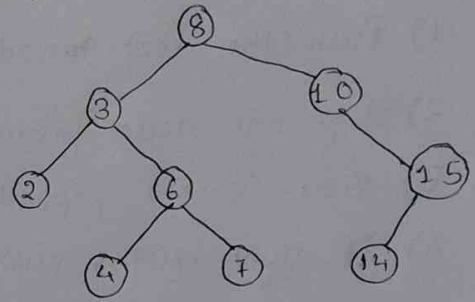
st.push(popped.right);

}

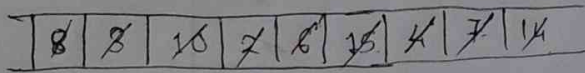
}

* BFS Traversal (Breadth-First Search)

- 1) Push node (root) on queue
- 2) Pop one node from queue
- 3) Visit Popped (print) node
- 4) if left exists, push it on queue
- 5) if right exists push it on queue
- 6) while queue is not empty, repeat steps (2-5)



Queue



Traversal : 8 3 10 2 6 15 4 7 14

* BST - Height

Height of root = Max (height of (left Sub tree), height of (right Sub tree))

- 1) If left or right subtree is absent return -1;
- 2) Find height of left subtree
- 3) Find height of right subtree.
- 4) Find max height
- 5) add one to max height and return.

```
int height (Node trav) {  
    if (trav == null)  
        return -1;  
    int h1 = height (trav.left);  
    int h2 = height (trav.right);  
    return 1 + max(h1, h2);  
}
```

* Capacity of Binary Tree:-

- max. number of nodes for given height.

h	n (nodes)
-1	0
0	1
1	3
2	7
3	15

$$\text{Capacity} = 2^{h+1} - 1$$

$$n = 2^{h+1} - 1$$

i.e. $2^h \approx n$

$$h \log 2 = \log n$$

$$h = \frac{\log n}{\log 2} \rightarrow \text{constant}$$

$$h = \log n$$

Time Complexities:-

① Add : $O(h) \approx O(\log n)$

② Delete : $O(h) \approx O(\log n)$

③ Search : $O(h) \approx O(\log n)$

④ Traversal : $O(n)$.

* Skewed BST have maximum height of i.e same as no. of elements.

* Time complexity of operations in skewed BST is $O(n)$.

* Balanced BST

- BST having as minimum height as possible.
- done to speed up the searching operation in BST.

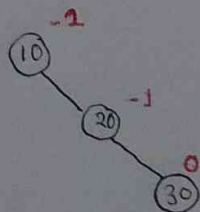
- Balance Factor:-

$$\text{BF (node)} = \text{Height (left sub tree)} - \text{Height (Right sub tree)}$$

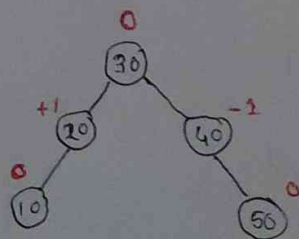
- Tree is balanced if, balance factors of all the nodes is either -1, 0, +1.

$$\text{balance factors} = \{-1, 0, +1\}$$

Ex:-



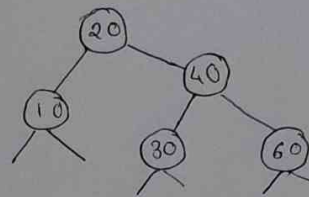
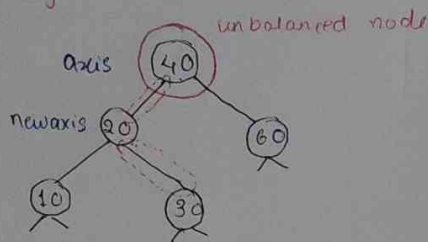
Not
Balanced



Balanced

* A tree can be balanced by applying series of left or right rotations on imbalance nodes.

* Right Rotation



Here, only 2 links got modified, left of 40 & right of 20

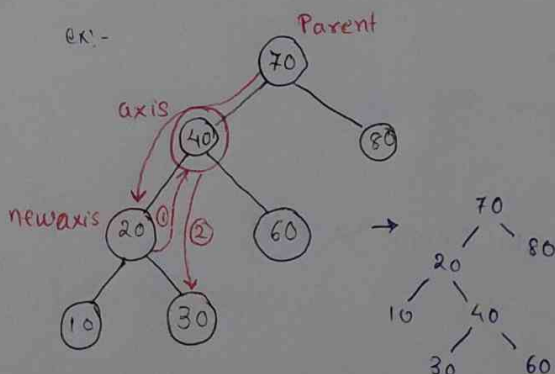
```

newaxis = axis.left;
axis.left = newaxis.right;
newaxis.right = axis;

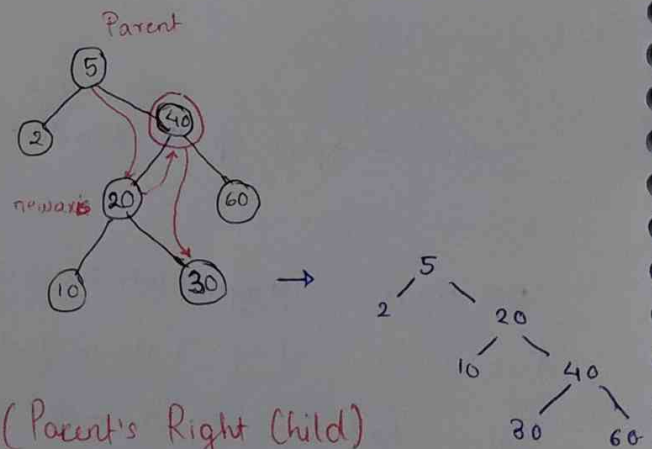
```

* Now, as we are changing the root, it might happen axis is left child or right child of the parent.

ex -



(Parent's left child)



(Parent's Right child)

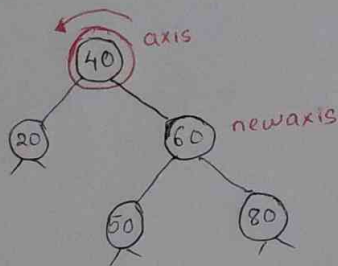
* Pseudo code :-

```

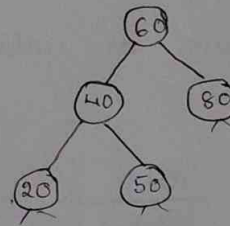
rightRotation(axis, parent) {
    newaxis = axis.left;
    axis.left = newaxis.right;
    newaxis.right = axis;
    if (axis == root) root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else
        parent.right = newaxis;
}

```


* Left Rotation



Left
Rotate



```
leftrotation (axis, parent) {
```

```
    newaxis = axis.right;
```

```
    axis.right = newaxis.left;
```

```
    newaxis.left = axis;
```

```
    if (root == axis) root = newaxis; // root is our axis
```

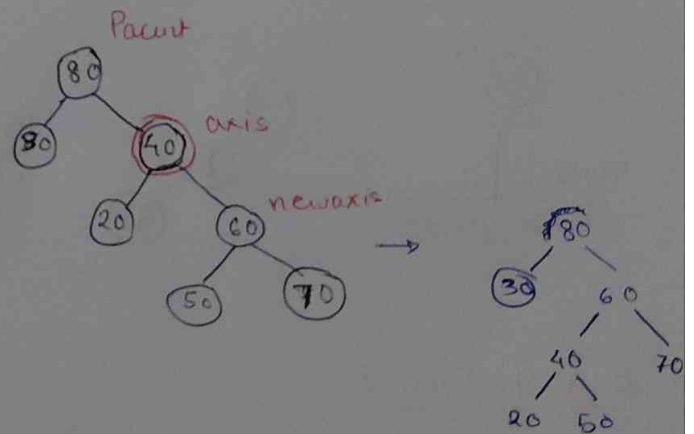
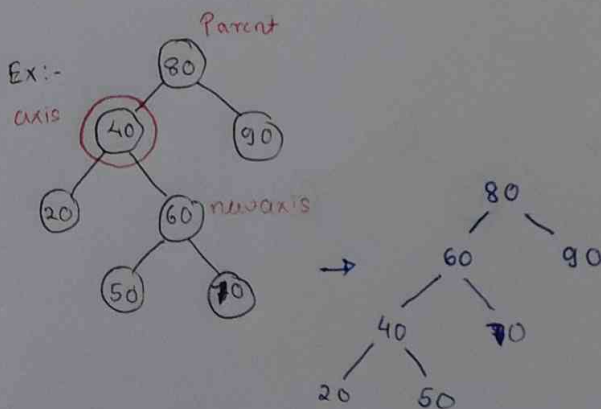
```
    else if (axis == parent.left) // axis is parent's left child
```

```
        parent.left = newaxis;
```

```
    else // axis is parent's right child.
```

```
        parent.right = newaxis;
```

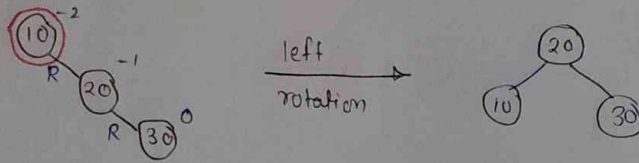
```
}
```



* RR Imbalance

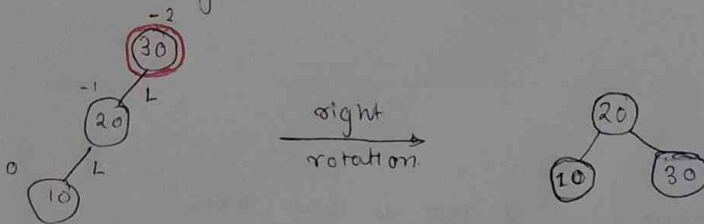
→ apply left rotation on imbalance node.

Keys :- 10, 20, 30



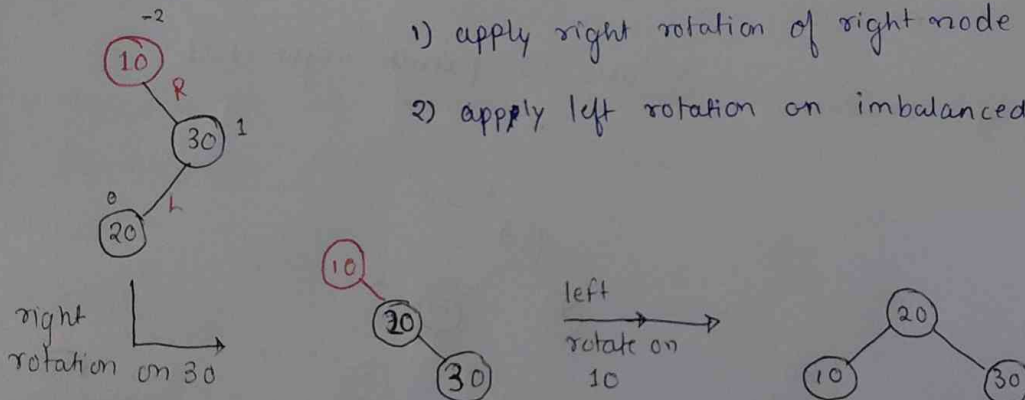
* LL Imbalance

→ apply right rotation on imbalance node.



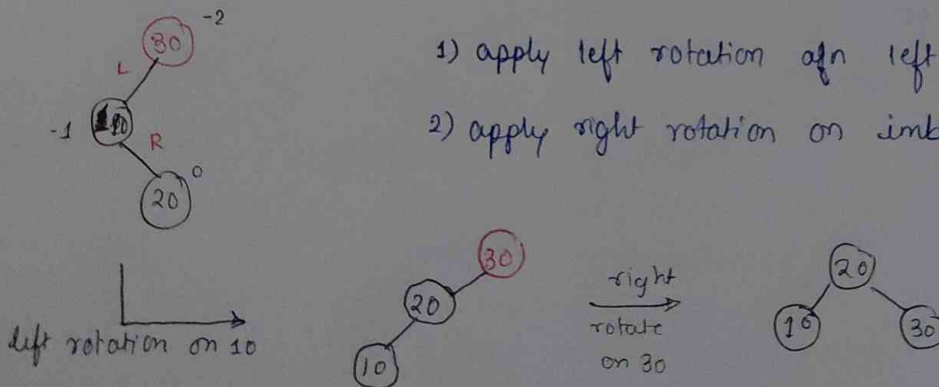
* RL Imbalance

- 1) apply right rotation of right node of imbalanced node.
- 2) apply left rotation on imbalanced node.



* LR Imbalance

- 1) apply left rotation of left node of imbalanced node.
- 2) apply right rotation on imbalanced node.



* 3 types of Balanced BST

- ① AVL Tree
- ② Red-Black Tree
- ③ Splay Tree

* AVL Tree

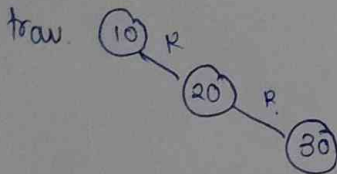
- Self balancing ~~balanced~~ binary Search Tree.
- On every insertion and deletion of a node, tree is getting balanced by applying rotations on imbalance nodes.
- The difference of ~~left~~ between heights of left & right subtree cannot be more than one for all nodes.
- Balance factors for all nodes are either $\{-1, 0, +1\}$.
- All operations of AVL Tree are performed in $O(\log n)$

Keys:- 40, 20, 10, 25, 30, 22, 50.

* How to check imbalance

① RR Imbalance.

Keys :- 10, 20, 30

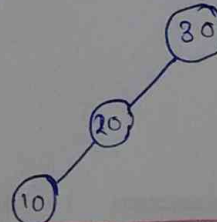


$$bf < -1$$

value > trav.right.data

② LL Imbalance.

Keys :- 30, 20, 10

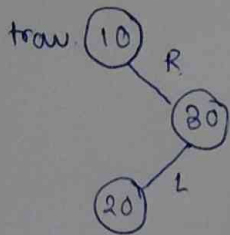


$$bf > 1$$

value < trav.left.data

③ RL Imbalance

Keys :- 10, 30, 20

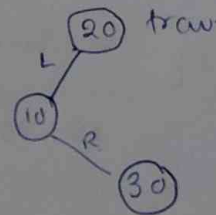


$$bf < -1$$

value > trav.right.data

④ LR Imbalance

Keys :- ~~20~~ 20, 10, 30

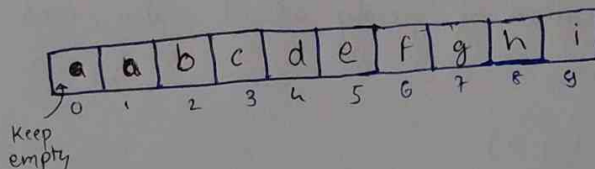
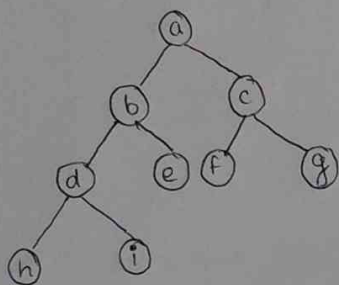


$$bf > 1$$

value < trav.left.data

* Complete Binary Tree

- All the levels of that tree should be completely filled except last level.
- All leaf nodes ~~at level h~~ must be at level h or $h-1$.
- All leaf nodes at level h must aligned as left as possible.
- Array implementation of complete binary tree is called as **Heap**.

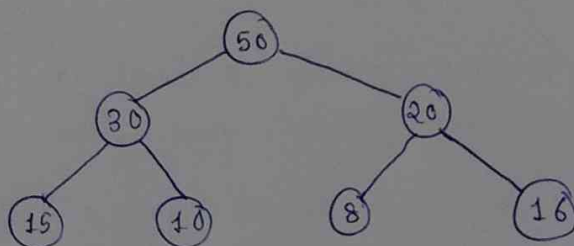


→ here array indices are used to maintain the parent-child relationship.

Node $\rightarrow i$
Parent $\rightarrow i/2$
left child $\rightarrow 2i$
right child $\rightarrow 2i+1$

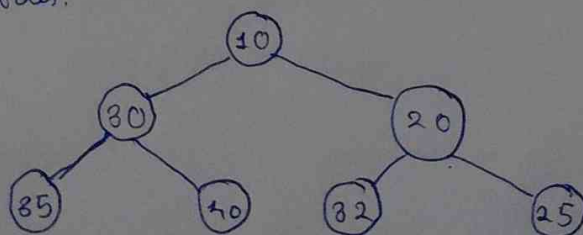
* Max Heap

→ a heap data structure in which each node is greater than both of its child.



* Min Heap

→ a heap data structure in which each node is smaller than both of its child nodes.



* Add data in heap

Algorithm :- 1) add value at 1st empty index from left side into array.
2) adjust position of newly added value by comparing it with all its ancestors one by one.

- to adjust position of newly added value, need to traverse from leaf to root positions.

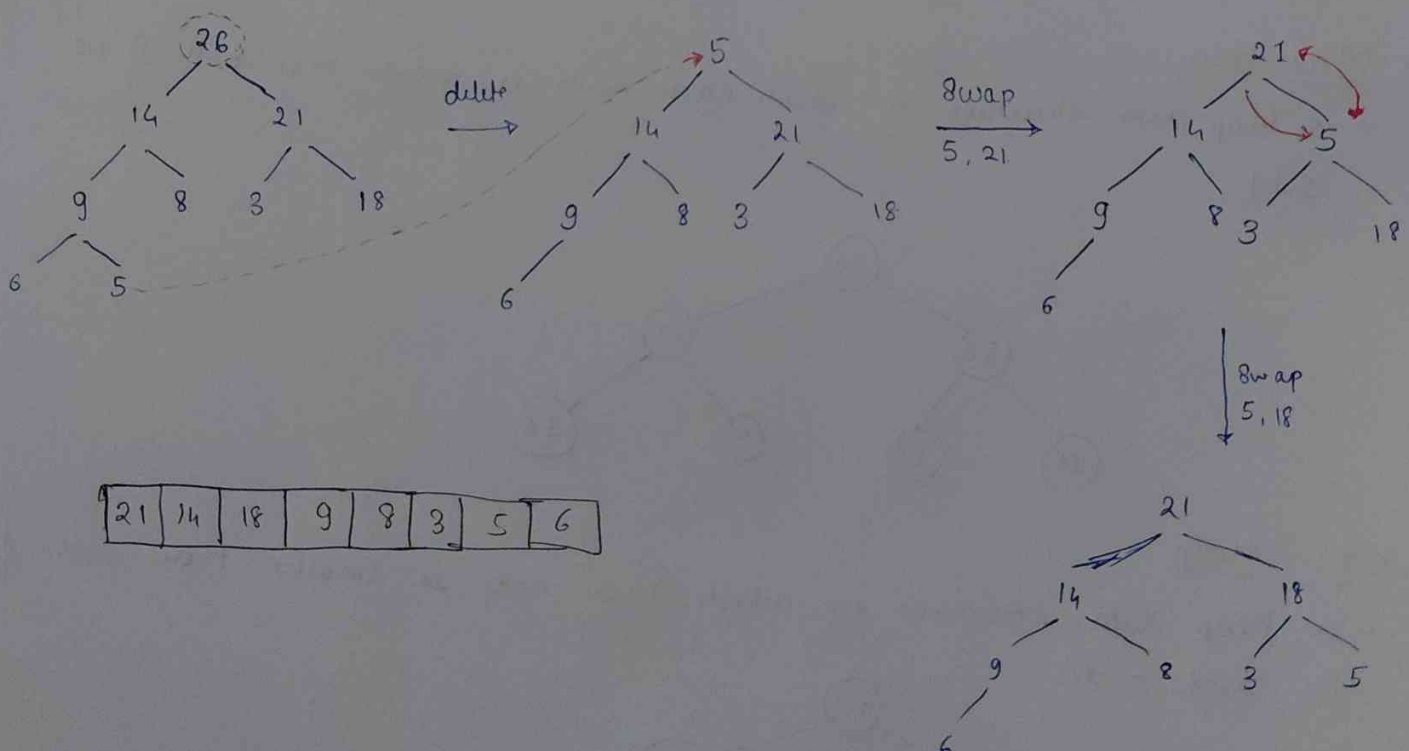
$$T(n) = O(\log n)$$

* Delete in heap

Property :- can delete only ~~from~~ root node from heap.

1) in max heap, always maximum element will be deleted from heap.

2) in min heap, always minimum element will be deleted from heap.



* Algorithm

- 1) Place last value of heap at root position.
 - 2) adjust position of new root by comparing it with all its descendants one by one.
- To adjust position of new root, need to traverse from root to leaf position.

$$T(n) = O(\log n)$$

* Priority Queue

- Queue where high priority data is always peeked or deleted.
- Priority can be implemented using array, linked list, heap data structures.
- An array is used to store a value and its associated priority. In some simpler implementations, the value itself might represent the priority (i.e. lower value means higher priority)
- An array and linked list implementation of priority queue often leads to inefficient implementation compared to heap based

* Ordered vs Unordered Array/Linked List

① Ordered Array/LL

- Elements are kept sorted by priority. Insertion requires shifting of existing members to maintain order, leading $O(n)$ time complexity.
- While deletion, the first element is our high priority element, so deletion requires $O(1)$ time complexity.

② UnOrdered Array/LL

- Elements are unsorted so we can add/insert the element at the last or anywhere in $O(1)$ time complexity.
- While deletion, the array is unsorted so we need to search for the element and then delete, which might result in $O(n)$ T.C.

Implementation

```
class Priority Queue {
```

```
    private int SIZE;
```

final

```
    private int heapSize;
```

```
    private int arr[];
```

```
    public PriorityQueue (int size) {
```

```
        SIZE = size;
```

```
        heapSize = 0;
```

```
        arr = new int [SIZE+1];
```

```
    }
```

```
    public boolean isEmpty () {
```

```
        return heapSize == 0;
```

```
    }
```

```
    public boolean isFull () {
```

```
        return heapSize == SIZE;
```

```
    }
```

```
    public int peek () {
```

```
        return arr [1];
```

```
    }
```

```
    public void addpush (int val) {
```

```
        if (isFull) return;
```

```
        heapSize++;
```

// put value of 1st empty index from left.

```
        arr [heapSize] = val;
```

// adjust position of newly added value by comparing it with parent.

```
        int pi = heapSize / 2, ci = heapSize;
```

```
        while (pi >= 1) {
```

```
            if (arr [pi] > arr [ci]) break;
```

// iterate till parent index is ~~not~~ greater than 1

```
            swap (arr [pi], arr [ci]);
```

// swap if parent < child.

```
            ci = pi;
```

// update indexes for next iteration

```
            pi = ci / 2;
```

```
        }
```



```

public void pop () {
    int
    // take backup of root element (max)
    int maxi = arr[1]
    // put last element in root / 1st index
    arr[1] = arr[heapsize];
    // after deletion, decrement heap size
    heapsize--;
    // adjust position of new root
    int pi = 1;
    int ci = 2 * pi; → index of left child node
    while (ci <= heapsize)
    {
        // find index of max child
        if (arr[ci] < arr[ci+1]) → if (ci+1 < heapsize && arr[ci+1] > arr[ci])
            ci = ci+1;

        // compare if parent > child, then break, else
        if (arr[pi] > arr[ci])
            break;

        swap(arr[pi], arr[ci]); // swap parent & child
        pi = ci;                // update indexes for next iteration
        ci = pi * 2;
    }
}

```

Heap Sort

1) create heap from given array.

asc $\leftarrow$ max min $\rightarrow$ desc.

2) delete all elements from heap one by one & place them last index of heap/arr.

Create heap - $n \log n$

Pseudocode :-

```
public class Heap {
```

```
    private int SIZE;
```

```
    public Heap (int size) {
```

```
        SIZE = size
```

```
    }
```

```
    public void deleteHeap (int arr[]) {
```

```
        // Swap first and last element
```

```
        for (i = 0; i < arr.length; i++) {
```

```
            int temp = arr[1];
```

```
            arr[1] = arr[SIZE];
```

```
            arr[SIZE] = temp;
```

```
            // decrement size of heap by 1.
```

```
            SIZE --;
```

```
            // adjust position of new root;
```

```
            int pi = 1;
```

```
            int ci = pi * 2;
```

```
            while (ci <= SIZE) {
```

```
                if ( (ci+1) <= SIZE && arr[ci+1] > arr[ci] ) // find index of max child.
```

```
                    ci = ci+1;
```

```
                if ( arr[pi] > arr[ci] ) // if parent is already > child.
```

```
                    break;
```

```
                // parent is less than max child.
```

```
                swap (arr[pi], arr[ci]);
```

```
                pi = ci;
```

for while

TC = $O(n \log n)$

```

public void heapify (int arr[]) {
    for (i = SIZE/2; i >= 1; i--) {
        int pi = i;
        int ci = pi * 2;
        while (ci <= SIZE) {
            // SAME AS DELETE HEAP
        }
    }
}

```

TC = $O(n)$

— } only these 2 changes

```

public void heapSort (int arr[]) {
    // create heap of given array
    heapify (arr);
    // delete all elements from heap
    deleteheap (arr);
}

```

TC = $O(n \log n)$

* Selection Sort

- 1) Select any position of the array
- 2) Find smallest out of remaining element
- 3) Swap selected position element with smallest element.
- 4) Repeat above steps until array is sorted.

TC = $O(n^2)$

Pseudocode:-

Solve (arr) {

```

for (i = 0; i < n-1; i++) {
    int curr = arr[i];
    int small = arr[i];
    for (j = i+1; j < n; j++) {
        small = min(small, arr[j]); if (arr[small] < arr[j])
        small = j;
    }
    if (curr >= arr[small]) {
        Swap (arr[i], arr[small]);
    }
}

```

```

for (i = 0; i < n-1; i++)
{
    int mini = i;
    for (j = i+1; j < n; j++)
    {
        if (arr[mini] > arr[j])
            mini = j;
    }
    if (arr[mini] <= arr[i])
        swap (arr[mini],
              arr[i]);
}

```

* When finding time complexity of searching & sorting algo, we count the number of comparisons we are doing, while iterating.

* Bubble Sort

- 1) Iterate till $n-1$ times & ~~check~~ for consecutive pair of elements.
- 2) if left ~~is~~ element is greater than right element, then swap.
- 3) do, this ~~n-1~~ times, till array is sorted.

To repeat $n-1$ times:-

$i = N-1$ ($i < N$)

i	j	So, condition will be
01	0-4	$j < N-i$
2	0-3	
3	0-2	
4	0-1	
5	0-0	

Solve (arr) {

```
for (i = 1; i < n; i++) {  
    for (int j = 0; j < n-i; j++) {  
        if (arr[j] > arr[j+1])  
            swap(arr[j], arr[j+1]);  
    }  
}
```

Improved

```
flag = 0;  
for (i = 1; i < n; i++) {  
    for (j = 0; j < n-i; j++) {  
        if (arr[j] > arr[j+1]) {  
            swap(arr[j], arr[j+1]);  
            flag = 1;  
        }  
    }  
    if (flag == 0)  
        break;  
}
```

TC = $O(n^2)$ → Avg/worst case

$O(n)$ → Best case

SC = $O(1)$

* Insertion Sort

- Pick one element of the array (start from 2nd index)
- Compare picked element with ~~rest~~ all its left neighbours one by one
- If left neighbour is greater, move it one position ahead.
- Insert picked element at its appropriate position.
- Repeat above ~~elements~~ steps until array is sorted. (n-1)

Pseudocode:-

* to pick elements : $i = 1 \rightarrow n-1$ ($i < n$)

* to compare with left neighbors $\rightarrow j = i-1 \rightarrow 0$ ($j \geq 0$)

Solve (arr) {

for ($i = 1$; $i < n$; $i++$) {

int temp = arr[i];

for ($j = i-1$; $j \geq 0$; $j--$) {

if (arr[j] > temp)

arr[j+1] = arr[j];

else

break;

}

arr[j+1] = temp; // insert at appropriate position

}

}

TC: $O(n^2)$ $\rightarrow$ worst/Avg

TC: $O(n)$ $\rightarrow$ Best case

SC: $O(1)$

* Merge Sort

- Divide array into two parts
- Sort both the partitions individually (by merge sort only)
- merge sorted partitions into temporary array.
- Overwrite temporary array into original array.

$$TC = O(n \log n)$$

$$SC = O(n)$$

```
mergeSort (arr, low, high)
```

```
{  
  if (low >= high) return;  
  mid = (low + high) / 2;  
  mergeSort (arr, low, mid);  
  mergeSort (arr, mid+1, high);  
  merge (arr, low, mid, high);  
}
```

```
merge (arr, low, mid, high) {
```

```
  temp = [];
```

```
  left = low, right = mid+1;
```

```
  while (left <= mid && right <= high) {
```

```
    if (arr[left] <= arr[right]) {
```

```
      temp.add(arr[left]);
```

```
      left++;
```

```
    }
```

```
    else {
```

```
      temp.add(arr[right]);
```

```
      right++;
```

```
    }
```

```
  }
```

```
  while (left <= mid) {
```

```
    temp.add(arr[left]);
```

```
    left++;
```

```
  }
```

```
  while (right <= mid) {
```

```
    temp.add(arr[right]);
```

```
    right++;
```

```
  }
```

// make changes in original array.

```
for (i = low → high) {  
    arr[i] = temp[i - low];  
}
```

$Tc = O(n \log n)$
 $Sc = O(n)$

* Quick Sort

- Select pivot/axis/reference element from array
- Arrange lesser elements on left side of pivot.
- arrange greater elements on right side of pivot.
- Sort left and right side of pivot again (by quick sort)

$Tc = O(n \log n)$
 $Sc = O(1)$

Pseudocode:-

```
public static void quickSort (int arr[], int left, int right) {
```

```
    if (left >= right) // stop if invalid or single element partition  
        return;
```

```
    // 1. Select pivot element from arr → can be 1st/last/mid elem
```

```
    // pivot = arr[left]; → no need to init
```

```
    // 2. arrange lesser elem on left & greater elem on right
```

```
    int i = left, j = right;
```

```
    while (i < j) {
```

```
        for (; arr[i] <= arr[left]; i++); // find element greater than pivot
```

```
        for (; arr[j] > arr[left]; j--); // find element lesser than pivot
```

```
        if (i < j) {
```

// swap ith elem with jth elem, when $i < j$

```
            int temp = arr[i];
```

```
            arr[i] = arr[j];
```

```
            arr[j] = temp;
```

```
        }
```

```
    }
```

// at last swap jth element with pivot

int temp = arr[left];

arr[left] = arr[j];

arr[j] = temp;

// sort right & left partition automatically.

quicksort(arr, left, j-1);

quicksort(arr, j+1, right);

}

→ No. of levels $\approx \log n$

→ No. of comparisons per level = n

→ Time Complexity of Quick sort is dependant on pivot.

→ Selection of pivot can be done by any of the above:-

i) extreme left

iii) middle

v) median of five

ii) extreme right

iv) median of three

vi) dual pivot

Graph

→ Graph is a non linear data structure having set of vertices (nodes) and a set of edges (arcs)

$$G = \{V, E\}$$

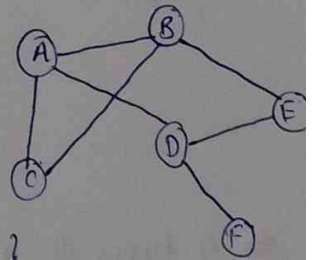
Where, V is set of vertices and E is set of edges.

* Vertex (node) is an element in the graph

$$V = \{A, B, C, D, E, F\}$$

* Edges (Arcs) is line connecting two nodes.

$$E = \{(A, B), (B, C), (C, A), (A, D), (B, E), (D, E), (D, F)\}$$



* Vertex A is set to be adjacent to B , if and only if there is an edge from A to B .

* Degree of Vertex :- Number of vertices adjacent to given vertex.

* Path :- Set of edges connecting any two vertices is called path b/w those two vertices.

$$\text{Path b/w } A \text{ \& } D = \{(A, B), (B, E), (E, D)\}$$

* Cycle :- Set of edges connecting ^{to} a node ~~to~~ itself is called as a cycle.

$$A \rightarrow \{(A, B), (B, E), (E, D), (D, A)\} :$$

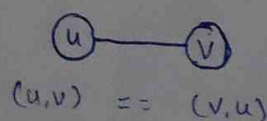
* loop :- An edge connecting node to itself is called as loop.

(loop is smallest cycle in graph)

* Undirected Graph

→ If we can represent any edge either (u, v) or (v, u) then it is referred as ~~an~~ unordered pair of vertices, i.e. undirected edge.

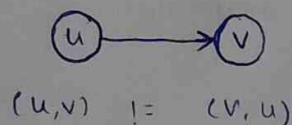
→ graph which contains undirected edges referred as undirected graph.



$$(u, v) == (v, u)$$

* Directed graph

- If we cannot represent any edge either (u, v) or (v, u) , then it is referred as an ordered pair of vertices, i.e. directed edge.
- graph which contains set of directed edges referred as directed graph or (digraph)
- graph in which each edge has some direction.



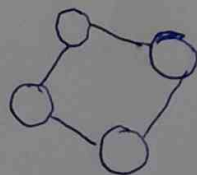
* 4 types of graph, based on weights

- ① undirected unweighted graph
- ② undirected weighted graph
- ③ directed unweighted graph
- ④ directed weighted graph

* Types of Graph

① Simple Graph

- having multiple edges b/w adjacent nodes and no loops

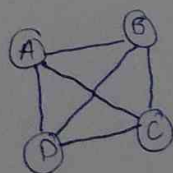


② Complete graph

- Simple graph in which each node is adjacent with every other node.

① undirected graph : No. of edges = $\frac{n(n-1)}{2}$

② directed graph : No. of edges = $n(n-1)$

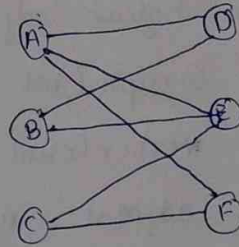


* ③ Connected Graph

- Simple graph in which there is some path exist b/w any two nodes/vertices.
- Can traverse the entire graph starting from any vertex.

④ Bipartite Graph

- vertices can be divided into two disjoint sets.
- vertices in first set are connected to vertices in second set.
- Vertices in a set are not directly connected to each other.

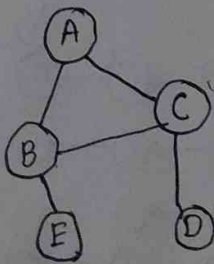


* Graph Implementation (Adjacency Matrix)

- If graph has V vertices, a $V \times V$ matrix can be formed to store edges of the graph.
- Each matrix element represent presence or absence of the edge b/w the vertices.
- For non-weighted graph, 1 indicate edge or 0 indicates no edge
- For weighted graph: weight of edge indicates presence of edge or else infinity sign (∞) represent presence of no-edge.

(In case of undirected graph, we always get symmetric matrix)

(In case of undirected graph every edge is entered twice, but in case of directed graph each edge is entered only once)



	A	B	C	D	E
A	0	1	1	0	0
B	1	0	1	0	1
C	1	1	0	1	0
D	0	0	1	0	0
E	0	1	0	0	0

Impl:-

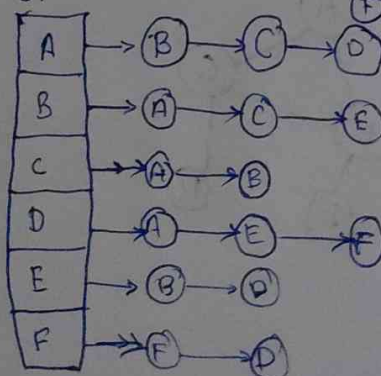
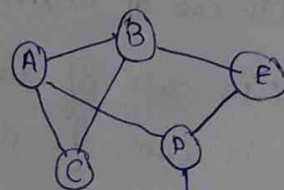
```

public class Graph {
    private int vertexCount;
    private int edgeCount;
    private boolean adjMat[][];
    public Graph(int v) {
        vertexCount = v;
        adjMat = new boolean[v][v];
    }
    public void accept(Scanner sc) {
        System.out.print("Enter edge count: ");
        edgeCount = sc.nextInt();
        System.out.print("Enter edge start & end: ");
        for (i = 0; i < edgeCount; i++) {
            start = sc.nextInt();
            end = sc.nextInt();
            adjMat[start][end] = true;
            adjMat[end][start] = true; // comment this for directed graph
        }
    }
}

```

* Adjacency list

- Each vertex holds list of adjacent vertices.
- For non-weighted graph, only neighbour vertices are stored.
- For weighted graph neighbor vertices and weights of connecting edges are stored.
- Space complexity of this implementation is $O(V+E)$
- If graph is sparse (with fewer no. of edges) this implementation is more efficient.



* Impl

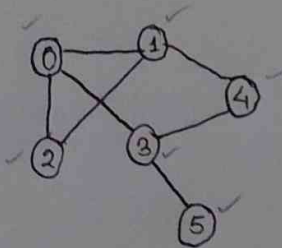
```

public class Graph {
    private int vertexCount;
    private int edgeCount;
    private List<Integer> adjList[];
    public Graph (int v) {
        vertexCount = v;
        adjList = new List<Integer>[v];
        for (i=0; i<v; i++)
            adjList[i] = new LinkedList<Integer>();
    }
}

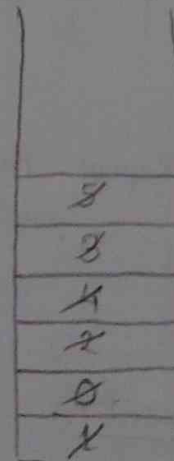
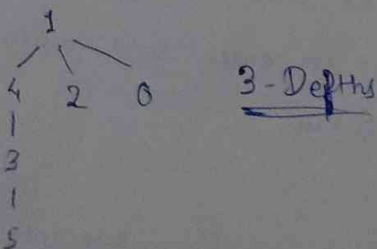
```

* DFS Traversal

- ① Choose a vertex as start vertex.
- ② Push start vertex on stack and mark it.
- ③ pop vertex from stack.
- ④ print the vertex
- ⑤ put all non-visited neighbors of the vertex on the stack & mark them
- ⑥ Repeat 3-5 until stack is empty.



Traversal: 1 0 4 3 5 2 0



```
public void DFSTraversal (int Start) {
```

```
// 0. create stack to push and array to mark vertices
```

```
Stack<Integer> st = new Stack<Integer> ();
```

```
boolean[] marked = new boolean [vertexCount];
```

```
// 1. Choose Start vertex - Start
```

```
// 2. push vertex on stack & push mark it
```

```
st.push (Start);
```

```
marked [Start] = true;
```

```
while (!st.empty()) {
```

```
// 3. pop vertex from stack
```

```
int u = st.pop();
```

```
// 4. visit & print the popped vertex
```

```
System.out (" " + u);
```

```
// 5. push all the non marked adjacents in stack and mark them
```

```
for (w = 0; w < vertexCount; w++) {
```

```
if (!marked [w] && adjMat [u][w]) {
```

```
st.push (w);
```

```
marked [w] = true;
```

```
}
```

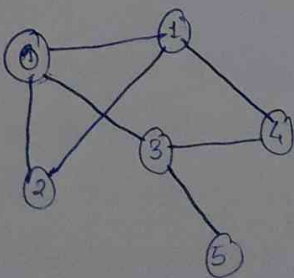
```
}
```

```
} // 6. Repeat this steps 3-5 till stack is empty.
```

```
System.out (" ");
```

```
}
```

* BFS Traversal



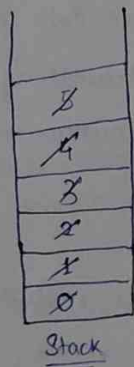
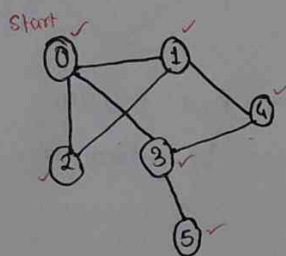
- Choose a vertex as start vertex
- push start vertex on queue and mark it.
- pop vertex from queue.
- print the vertex
- put all non visited neighbors into the queue and mark them.
- Repeat 3-5 until queue is empty

- * BFS is used for searching of nearest elements. (Shortest Path)
- * DFS is used for searching for farthest elements.

* Spanning Tree

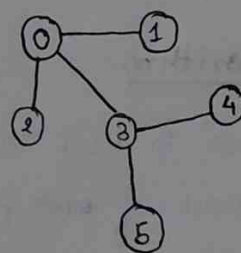
- Tree ^{is graph} without cycles. Includes all vertices (V) and edges (V-1).
- * Spanning tree is connected sub-graph of the given graph that contains all the vertices and sub-set of edges.
- * Spanning tree can be created by removing few edges from the graph which are causing cycles to form.
- One graph can have multiple ^{different} spanning trees.

* Using DFS



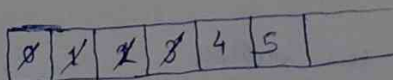
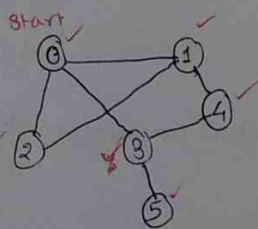
Edges

0-1, 0-2, 0-3
3-4, 3-5



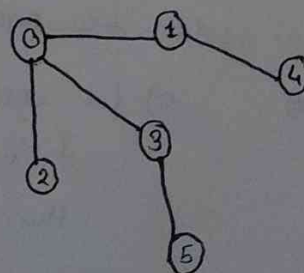
DFS
(Spanning Tree)

* Using BFS



Edges:-

0-1, 0-2, 0-3,
1-4, 3-5



(BFS Spanning Tree)

* In case of weighted graph, a spanning tree can be made with minimum weight (sum of weights).

Such tree is called minimum spanning tree.

* Spanning tree can be found using :-

- ① BFS Spanning Tree
- ② DFS Spanning Tree
- ③ Prim's MST
- ④ Kruskal's MST

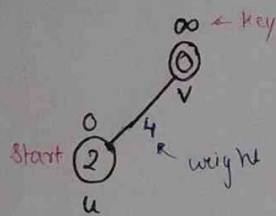
MST applications:- ① in networking to detect loops in router.

② to find minimum/shortest path in a city to deploy water pipeline/fibre optic cable.

③ Can be used in travelling application to find travelling plans based on minimum cost/distance b/w two cities.

* Prim's Algorithm

- 1) Create a mst to keep track of vertices included in MST.
- 2) Also keep track of ~~each~~ parent of each vertex. Initialize parent of each vertex to -1.
- 3) Assign a key to all vertices in input graph. Key for all vertices should be initialized to ∞ (∞). The start key should be 0.
- 4) While mst doesn't include all the vertices
 - i) Pick a vertex u which is not there in mst and has minimum key.
 - ii) Include vertex u to mst.
 - iii) update key and parent of all adjacent vertices of u .
 - a) for each adjacent vertex v .
 - if weight of edge $u-v$ is less than the ~~current~~ current key of v , then update the key as weight of $u-v$.
 - b) Record u as parent of v .



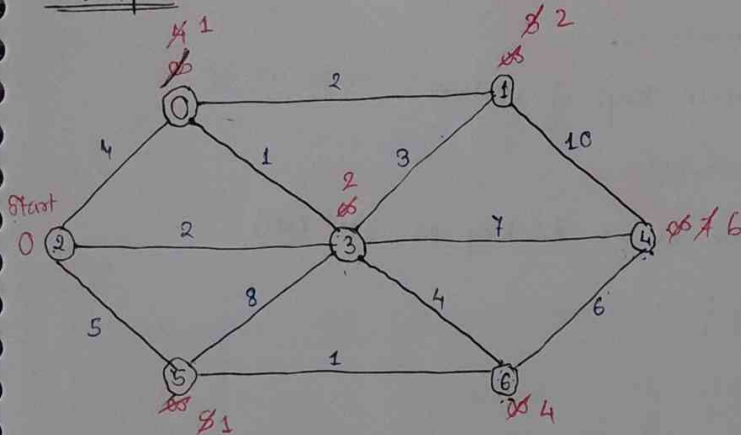
if (weight(u-v) < keys[v])
 keys[v] = weight(u-v)

(weight(u-v) = adj[u][v])

88.

make u the parent of v

Example



i	k	P
0	1 1	2 3
1	2 2	3 0
2	0	-1
3	2 2	1 2
4	6 6	3 6
5	1 1	2 5
6	4 4	3 4

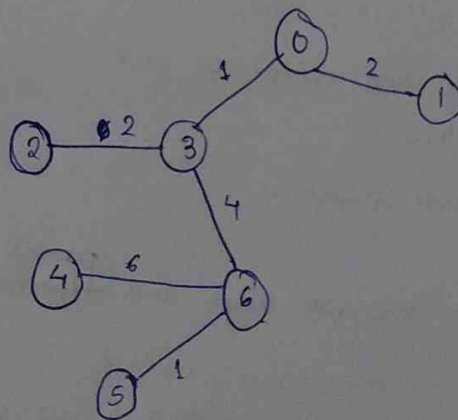
Choice :-

- 1) 2 → 0, 3, 5 (2nd choice is vertex with minimum key except 2)
- 2) 3 → 0, 1, 4, 5, 6 (3rd choice is vertex with minimum key except 3 & 2)
- 3) 0 → 1 (4th choice is vertex with minimum key from mst except 3 & 2 & 1)
- 4) 1 → 4 (not updated as 10 < 7 false)
- 5) 6 → 4, 5
- 6) 5 → none (∵ all adjacents are already present in mst)
- 7) 4 → none (— — —)

* Note :-

if we have unique weights then we can get only one MST.

And if weights are duplicate then we can get multiple diff MST



Pmpl.:-

```
public void primsMST (int start) {
```

```
//1. create a set to keep track of vertices in MST.
```

```
boolean mst[] = new boolean [vertexCount];
```

```
//2. create an array to keep track of array parents
```

```
int parents[] = new int [vertexCount];
```

```
//3. create an array to maintain keys of vertices.
```

```
int keys[] = new int [vertexCount];
```

```
//4. Set parent of each vertex as -1 & key as INF ( $\infty$ )
```

```
for (i=0; i < vertexCount; i++) {
```

```
    parents[i] = -1;
```

```
    keys[i] = INF;
```

```
    mst[i] = false;
```

```
}
```

```
//5. make key of start vertex to 0.
```

```
keys[start] = 0;
```

```
//6. While all vertices are not added in mst
```

```
int count = 0;
```

```
while (count < vertexCount) {
```

```
    //6.1. pick min key vertex which is not present in mst.
```

```
    int u = findMinkey (keys, mst);
```

```
    //6.2. add picked vertex in mst
```

```
    mst[u] = true;
```

```
    //6.3. increment count
```

```
    count++;
```

```
    //6.4. update the new keys and parentadjacents of adjacents.
```

```
    for (v=0; v < vertexCount; v++) {
```

```
        if (!mst[v] && adj[u][v] != INF && adj[u][v] < keys[v]) {
```

```
            keys[v] = adj[u][v];
```

```
            parents[v] = u;
```

```

* int findminkey ( mst, & keys) {
    int mini = INT_MAX;
    int index = -1;
    for ( i=0; i < keys.length; i++) {
        if ( !mst[i] && mini < keys[i]) {
            mini = keys[i];
            index = i;
        }
    }
    return index;
}

```

* Dijkstra's Algorithm (Single source shortest path algorithm)

Algo :-

- 1) create a set spt to keep track of vertices included in shortest path tree.
- 2) Track distance of all vertices in the input graph. Distance for all vertices should be initially INF (∞). The start _{vertex} distance of ~~it~~ should be 0.
- 3) while spt doesn't include all the vertices.

a) pick a vertex u , which is not there in spt and has minimum distance.

b) include vertex u to spt.

c) update distances of all adjacent vertices of u .

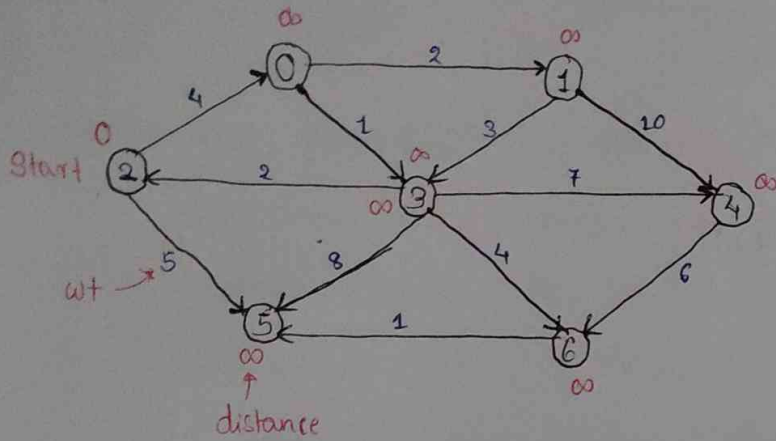
for each adjacent vertex v ,

if (distance of u + weight of edge $u-v$) is less than the current distance of v , then update its distance as (distance of u + weight of v)

if $(d(u) + w(u,v) < d(v))$ ~~then~~

$$d[v] = d[u] + w(u,v)$$

$$(w(u,v) = \text{adj}[u][v])$$



	D
0	$\infty \rightarrow 4$
1	$\infty \rightarrow 6$
2	0
3	$\infty \rightarrow 5$
4	$\infty \rightarrow 12$
5	$\infty \rightarrow 5$
6	$\infty \rightarrow 9$

Vertex pick:-

2 $\rightarrow$ 0 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 1 $\rightarrow$ 6 $\rightarrow$ 4



$\because$ 3 & 5 has same weights so we can choose anyone from both.

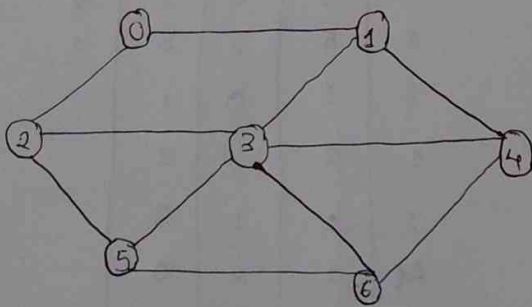
* Kruskal's Algorithm

1) Sort all edges in ascending order of their weight.

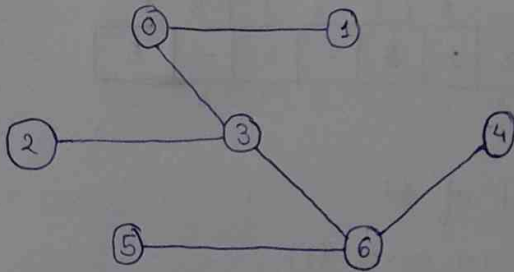
2) Pick the smallest edge.

- Check if it forms cycle with the spanning tree formed so far.
If cycle is not formed, including this edge.
else discard it.

3) Repeat Step 2, until there are $(v-1)$ edges ~~are~~ in the spanning tree.



↓ Spanning tree



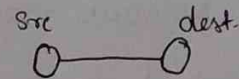
s	d	wt	
0	3	- 1	✓
6	5	- 1	✓
0	1	- 2	✓
3	2	- 2	✓
1	3	- 3	✗
2	0	- 4	✗
3	6	- 4	✓
2	5	- 5	✗
4	6	- 6	✓
3	4	- 7	
3	5	- 8	
1	4	- 10	

Diff b/w Kruskal's and Prim's Algorithm.

* Union find Algorithm (used to find cycle in a graph/mst)

- ① Consider all vertices as disjoint sets (parent = -1)
- ② For each edge in graph/mst

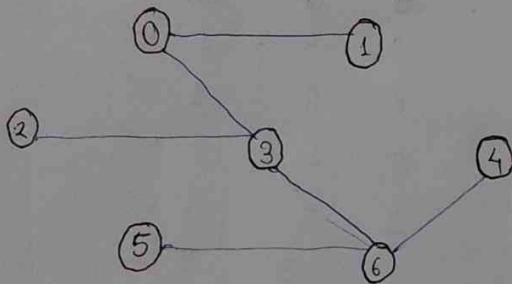
a) Find Set(root) of first vertex (Source)



b) Find Set(root) of second vertex (dest)

c) If both are in same set (same root), cycle is detected

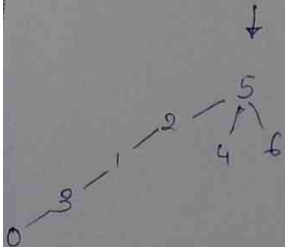
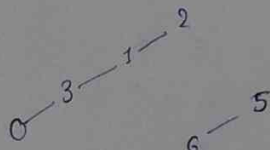
d) Otherwise, merge (Union) both sets, i.e. add root of first set under second set.



sr	dr	s	d	wt
0	3	0	3	1
6	5	6	5	1
3	1	0	1	
1	2	3	2	
2	2	1	3	
2	2	2	0	
2	5	3	6	
5	5	2	5	
4	5	4	6	
5	5	3	4	
5	3	3	5	

Parents

3	2	5	1	5	-1	5
0	1	2	3	4	5	6



* To find root of set

```

find (int v, int parent[])
{
    while (parent[v] != -1) {
        v = parent[v];
    }
    return v;
}

```

* To find union

```

void union (int sr, int dr, int parent[])
{
    parent[sr] = dr;
}

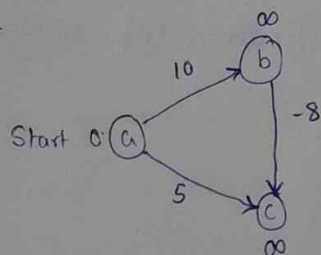
```

* Dijkstra's Algorithm may fail on negative edges.

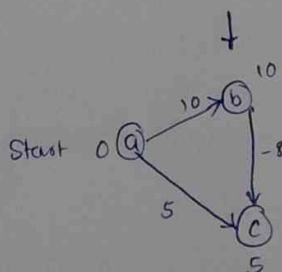
for this, we use Bellman-Ford Algorithm to counter the failure of on -ve edges.

(In bellman-ford, we ~~will~~ update the distance $V-1$ times for each edge)

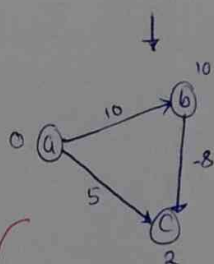
Ex:-



- 1) $0 + 10 < \infty \rightarrow \text{update}$
- 2) $0 + 5 < \infty \rightarrow \text{update}$
- 3) $\infty + (-8) < \infty \rightarrow \text{false} \rightarrow \text{no update}$



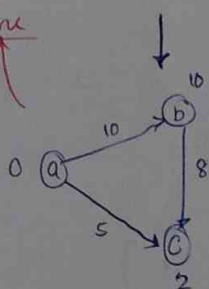
- 1) $0 + 10 < 10$ false (no update)
- 2) $0 + 5 < 5$ false (no update)
- 3) $10 + (-8) < 5$ True update.



(Here, is the final graph after updating weights $n-1$ times)

(But bellman algo. says, check one more time and if the final graph also has same weights, then answer is correct & else it is incorrect)

Weights Same



* Floyd-Warshall Algorithm (All pair shortest path algo)

1) Create distance matrix to keep distance of every vertex from each vertex.

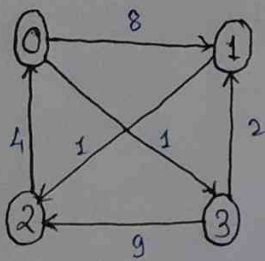
- Initially assign it weights of all edges among vertices.

(i.e. adjacency matrix)

2) Consider each vertex (i) in b/w pair of any two vertices (u,v) and find the optimal distance b/w source (u) & destination (v) considering intermediate vertex.

i.e. if $(\text{dist}[u][i] + \text{dist}[i][v]) < \text{dist}[u][v]$

$$\text{dist}[u][v] = \text{dist}[u][i] + \text{dist}[i][v]$$



$$\text{adj} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} \infty & 8 & \infty & 1 \\ \infty & \infty & 1 & \infty \\ 4 & \infty & \infty & \infty \\ \infty & 2 & 9 & \infty \end{bmatrix} \end{matrix}$$

$$d = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & \infty & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

distance matrix

diagonals are made "0" as distance of a node from itself is 0.

↓

$$d_0 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

↓
using intermediate

$$d_1 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$d_2 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

→

$$d_3 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

↓
final matrix

* Pseudocode :-

```
int dist[vcount][vcount];
for (u=0; u<vcount; u++) {
    for (v=0; v<vcount; v++) {
        dist[u][v] = adj[u][v];
    }
    dist[u][u] = 0;
}
```

```
for (i=0; i<vcount; i++) {
    for (u=0; u<vcount; u++) {
        for (v=0; v<vcount; v++) {
            if (dist[u][i] + dist[i][v] < dist[u][v]) {
                dist[u][v] = dist[u][i] + dist[i][v];
            }
        }
    }
}
```

Tc = $O(n^3)$